



Development of a Digital
PID Position Controller for
a Two-Arm Robotic System
Using DC Servo Motors and
a Raspberry Pi Pico

Project B

Development of a Digital PID Position Controller for a Two-Arm Robotic System Using DC Servo Motors and a Raspberry Pi Pico

Peiman Edalatjoo - Student No 94112 – Summer 2025 semester

HKA – Hochschule Karlsruhe Technik und Wirtschaft

Mentor: Professor Samuel E. de Lucena

Co-Mentor: Professor Thorsten Leize

Hochschule Karlsruhe
University of
Applied Sciences



2-Arm Robotic Dc Motors

Peiman.edalatjoo@proton.me

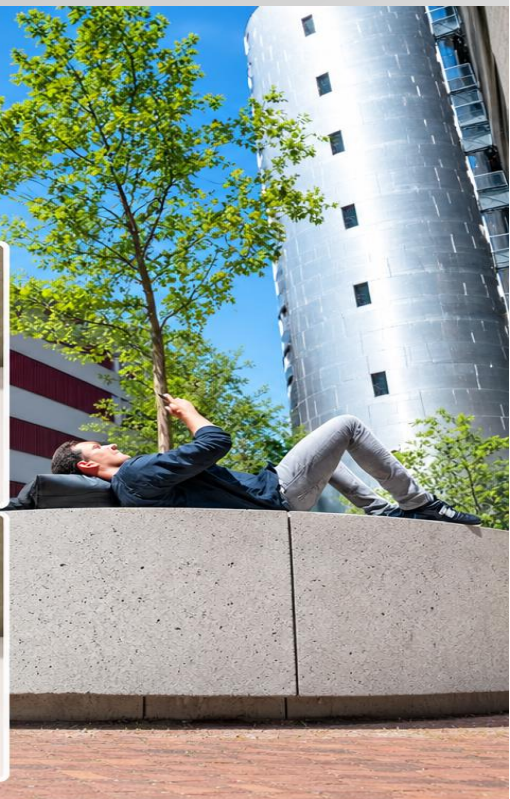
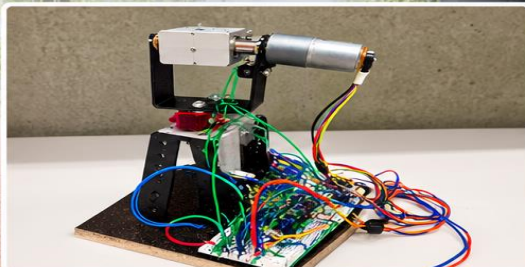
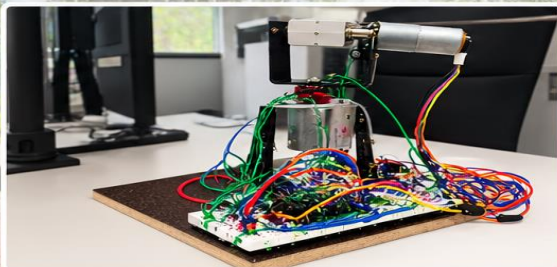
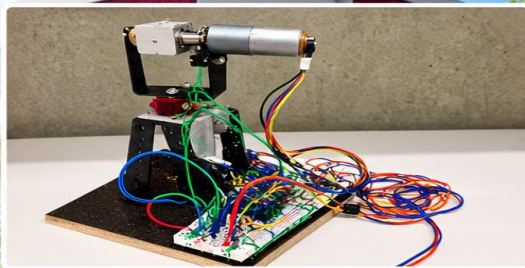
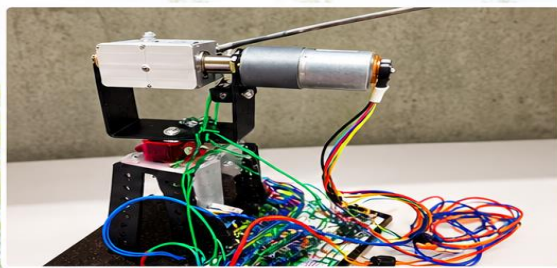


Table of content:

Introduction	4
2.....	5
System Overview	5
3.....	8
Hardware Design	8
3.1 Main Components.....	8
3.2 Connections.....	10
3.3 Encoder	11
3.4 Zero Switch Signals	21
4. Software Architecture	22
4.1 Main Program (main.cpp).....	24
4.2 Motor Abstraction (motor.*).....	27
4.3 Zero Interpolation (zero_calibration.*).....	28
4.4 Safety and Watchdog Logic (system_safety.*)	28
5.....	29
Control and Operating Modes.....	29
5.1 Modes.....	29
5.2 PID Position Control.....	30
6. Safety Concept.....	39
6.1 Emergency push button.....	40
6.2 Zero Switch as Collision / Limit Switch.....	41
6.3 Watchdogs.....	42
7. User Interface and Operation	42
7.1 Command Syntax.....	42
7.2 Typical Operating Procedure	43
8. Conclusion and Future Work	43
9. Appendices.....	44
10. References.....	76

Introduction

This project implements a two-axis DC servo positioning system using a Raspberry Pi Pico microcontroller. Each motor is equipped with an incremental quadrature encoder and controlled by a PID algorithm to reach a commanded angular position.

Although many high-performance industrial MCUs provide dedicated encoder interfaces and fast control peripherals, the Raspberry Pi RP2040, with its dual Cortex-M0+ cores running at 133 MHz, is well suited for low-cost, multi-axis, and educational motion-control applications. In this project, one core is used to control two motors in parallel, achieving fast and precise positioning, with sufficient headroom for additional axes.

Industrial MCUs such as the Texas Instruments C2000 series (e.g., TMS320F2838x, F28003x) and the STMicroelectronics STM32G4, STM32F7, and STM32H7 series are common compact solutions for position-control applications.

The main goal of this project is to develop a small yet realistic industrial motion-control example. The focus is not only on positioning accuracy, but also on safe behavior under abnormal conditions such as wiring faults, emergency stops, and mechanical collisions.

The key objectives are:

- Control two DC motors with encoder feedback.
- Support different operating modes (manual, parallel, sequential).
- Implement zero interpolation to define a precise reference position.
- Integrate safety mechanisms: emergency switch, zero-switch collision detection, and watchdogs.
- Use a clean and modular software structure suitable for future extensions.

The final code is a collection of several development programs. Although separate programs were created for specific tasks such as PID tuning and velocity control experiments, their key functionalities have been consolidated into a single final code through configurable options in `motor_config.h`. These options include PID control modes (position or velocity), multiple zero-search methods, optional software filtering, and selectable debug outputs.

This report describes the hardware and software design in a clear, structured, and simple manner. The developed code is reusable for controlling multiple motors because it is based on a motor class, allowing multiple motor objects to be created from the same class. However, in this project, the code was tested with only two motors for the robotic arm. The final results were satisfactory and were achieved through numerous trials, failures, and design iterations.

During development, multiple code versions were created to address motor-related challenges such as positioning accuracy, vibration, and interrupt handling on the Raspberry Pi Pico, as well as user interaction. This included receiving PID parameters (K_p , K_i/T_i , K_d/T_d) or speed commands via keyboard, similar to G-code in NC/CNC systems, with checks for invalid inputs and error messages.

For simplicity, calibration and interaction features were removed from the final version, as they can be easily reimplemented and adapted to specific applications. Manual control via a potentiometer, previously implemented, was also excluded.

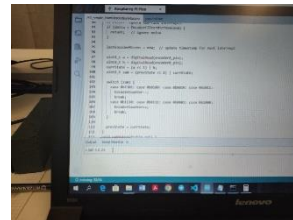
Therefore, this report focuses on the main code components required for safe and reliable motor operation, including the PID control loop, zero-position search, and emergency and safety systems. These parts are especially important because software errors could cause serious or irreversible damage, particularly when the system is scaled to higher power Motors or more expensive equipment. Brief comments are included in the code to explain the main functions and logic. The major system components are described separately in more detail. Some practical industrial considerations are also discussed, even though they are not implemented in the current project.

2

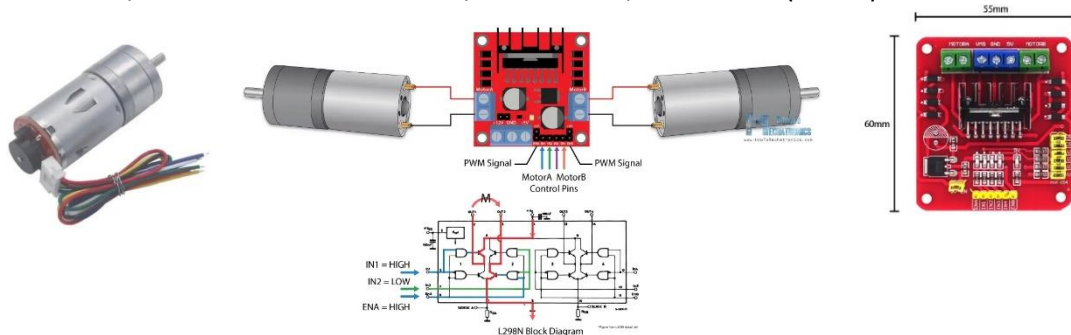
System Overview

The system consists of three main parts:

- A host PC running a serial terminal.
- A Raspberry Pi Pico microcontroller board running the control firmware.



- Two DC servo axes, each with a motor driver, an encoder, and a zero (home) switch.



user sends commands from the PC over USB serial to the Pico. The Pico interprets these commands, computes the motor outputs, and drives the motors through the motor drivers. The encoders provide feedback so that the controller can measure the current position. Zero switches are used to find a known reference position when the system start up. An additional emergency switch allows safe stopping of the system.

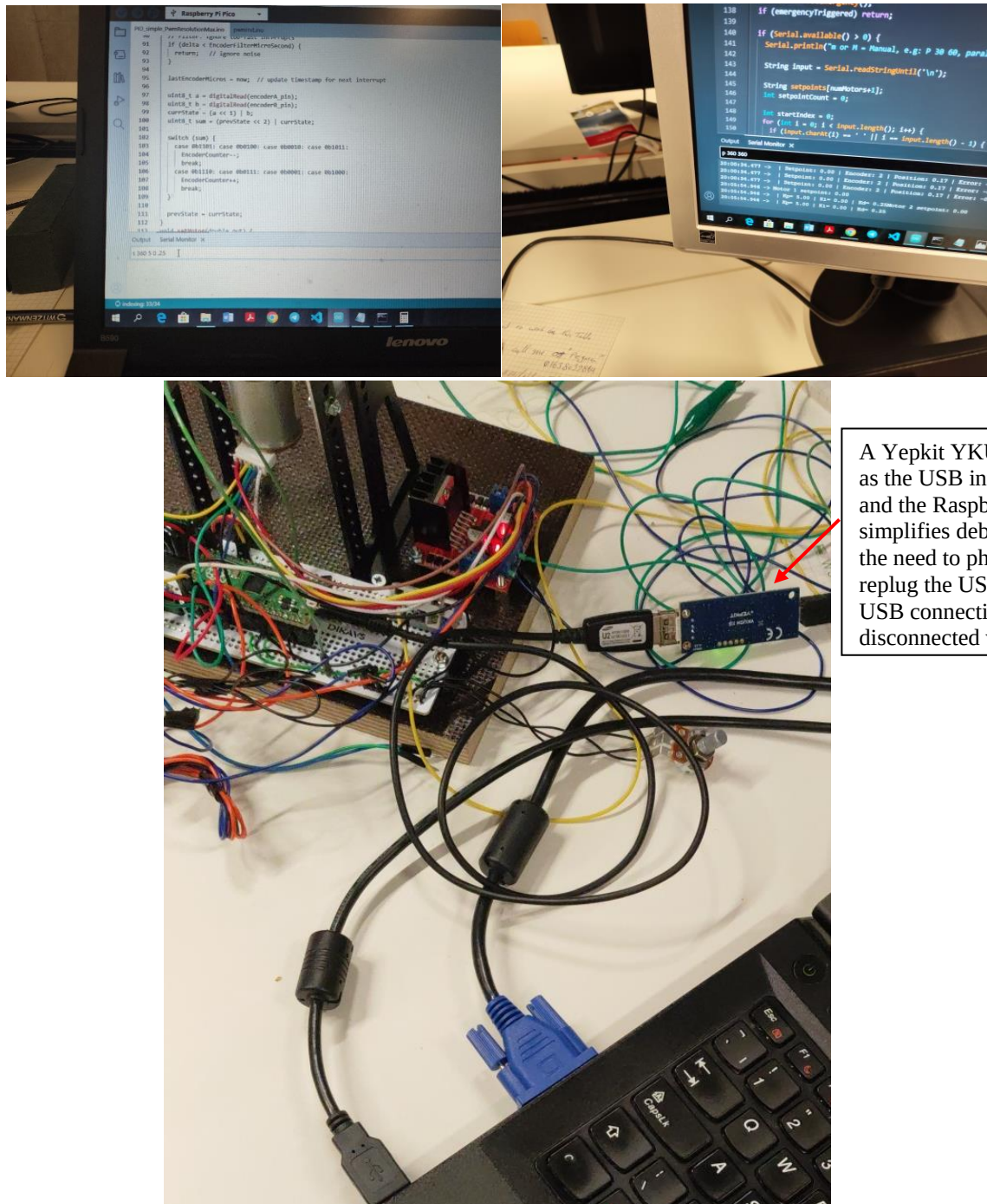


Figure 1

A Yekit YKUSH XS kit was used as the USB interface between the PC and the Raspberry Pi Pico. It simplifies debugging by eliminating the need to physically unplug and replug the USB connection. The USB connection can be connected or disconnected via command-line

Block diagram of the complete system (PC → Pico → motor drivers → motors, encoders, zero switches, emergency switch).

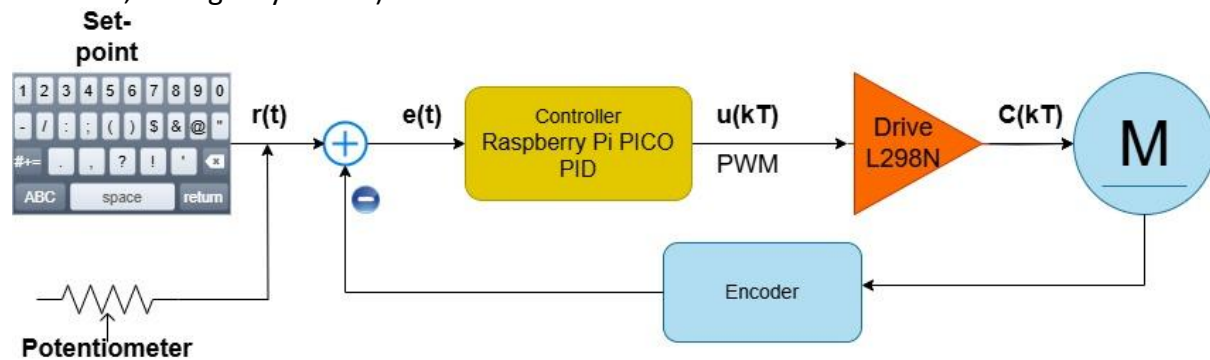


Figure 2

The typical operating sequence is:

1. Power on the system.
2. The Pico initializes all hardware and runs a zero interpolation routine (if zero switches are installed).
3. The system reports "System ready" on the serial terminal.
4. The user sends motion commands via serial (manual, parallel, or sequential mode).
5. The motors move to the requested positions while safety mechanisms continuously monitor the system.
6. In case of an emergency or collision, the system stops the motors and requires a new zero interpolation before normal operation is resumed.

Hardware Design

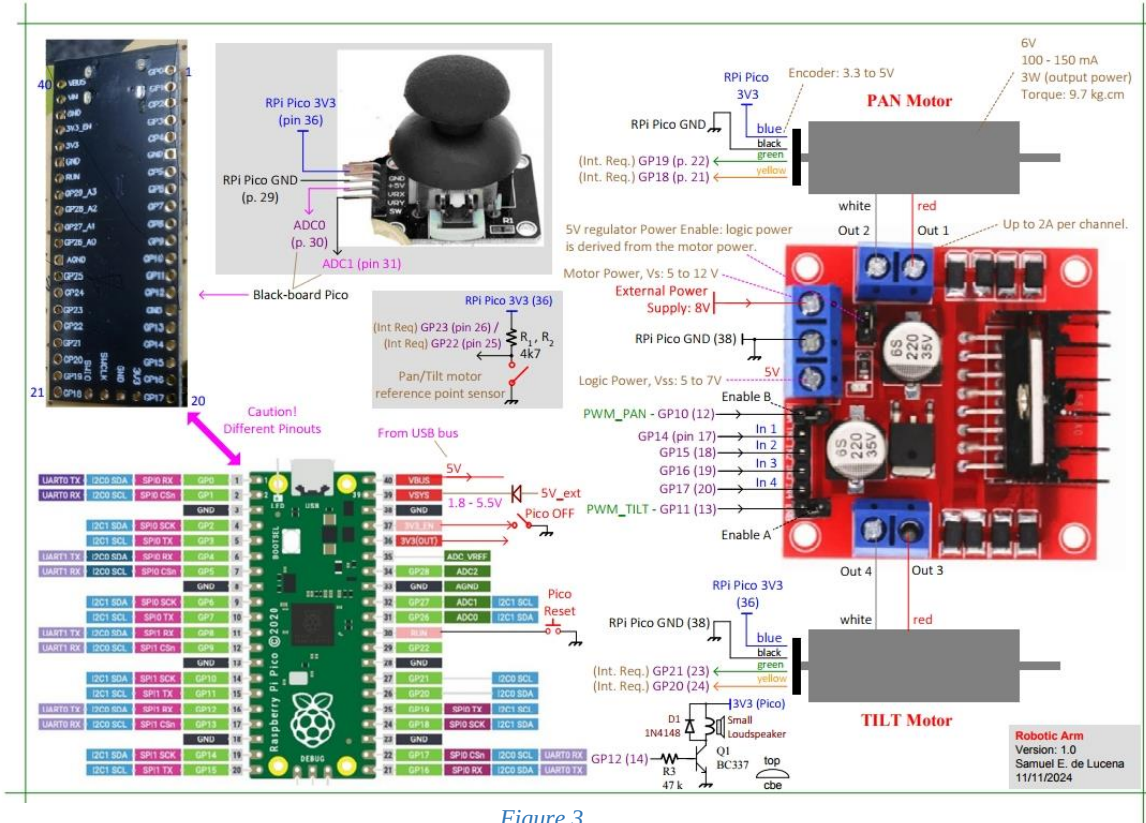


Figure 3

3.1 Main Components

The main hardware components of the system are:

- Raspberry Pi Pico microcontroller.
- Two DC motors with encoder feedback.
- Motor driver modules for each motor.
- Incremental encoders (one per motor).
- Zero (home) switches for each axis.
- Emergency stop push button with external pull-up resistor.
- Power supply for motors and logic.

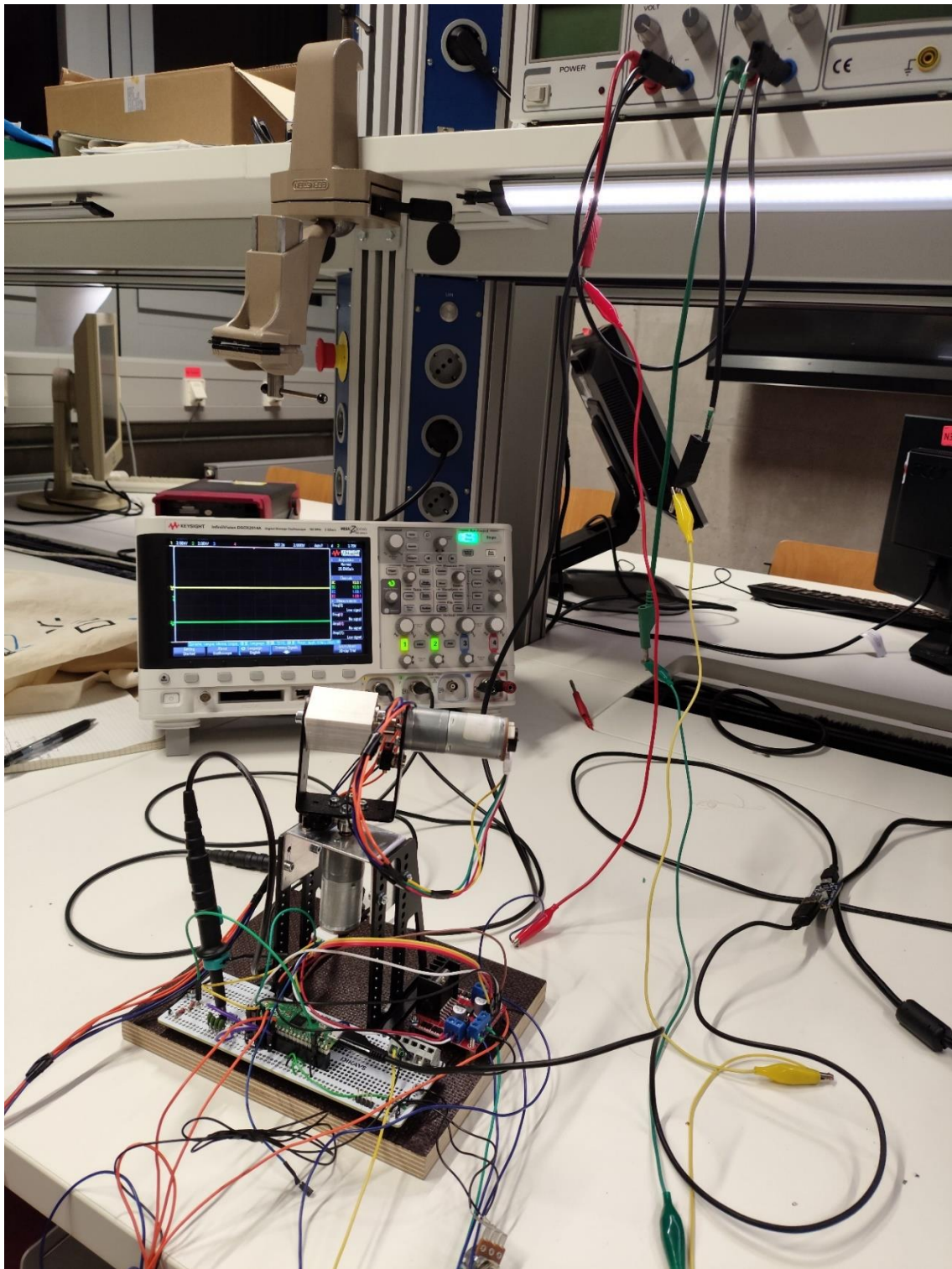


Figure 4

Figure 5



Challenges:

- Ordered parts didn't fit or match together.

Here in this picture you can see that the holes of Motor base holder and motor are not aligned. New holder and new holes on the Motors made.

Or here you can see a new holes-screws on motor disc and flange has been created in order to align with motor shaft and tight it.

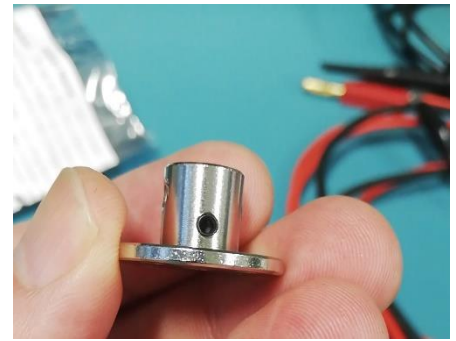


Figure 6

3.2 Connections

For details of connection please refer to Figure 3. Also below circuits has been added to the that figure.

- One zero switch for each Motor is connected as a digital input.

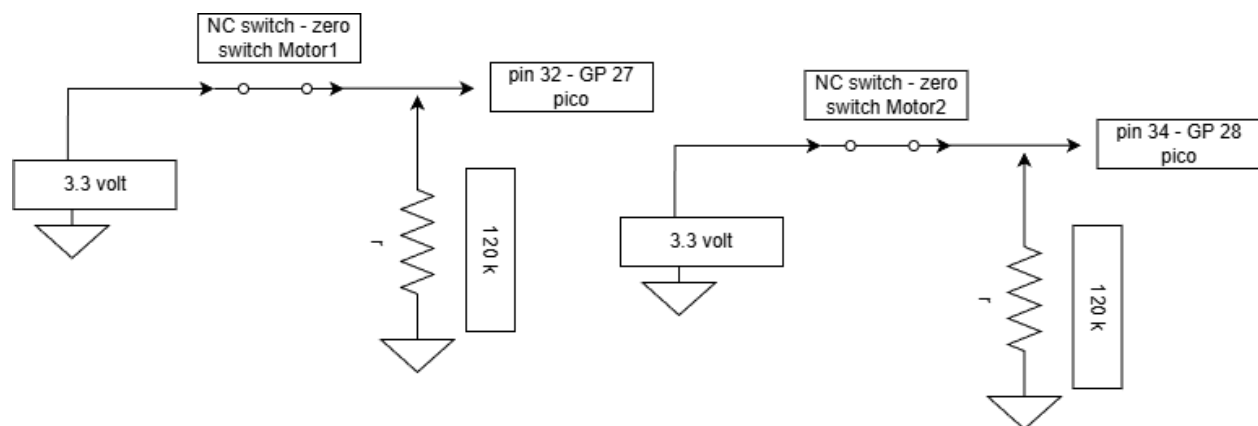


Figure 7

- An emergency push button is connected to a digital input using an external 120 kΩ pull-up or pull-down resistor. In this project, a pull-up configuration was usually used, which is acceptable because the circuit is closed in normal operation and referenced to ground. When the input is at ground level, the system operates normally; if the circuit is opened

(fail-safe condition), the input goes high via the pull-up resistor, and the system enters the emergency state.

However, I believe in practical industrial environments, a pull-down configuration with active-high monitoring should be preferred. If a wire is damaged and shorts to the equipment chassis or ground, the signal may remain permanently low, preventing detection of an emergency condition. Monitoring a driven high level (Vcc) provides better fault detection in such cases.

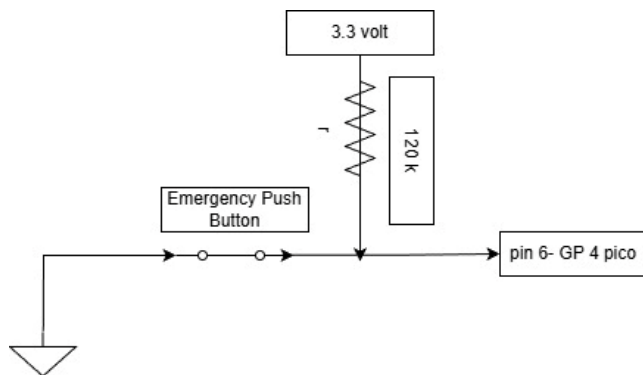


Figure 8

3.3 Encoder

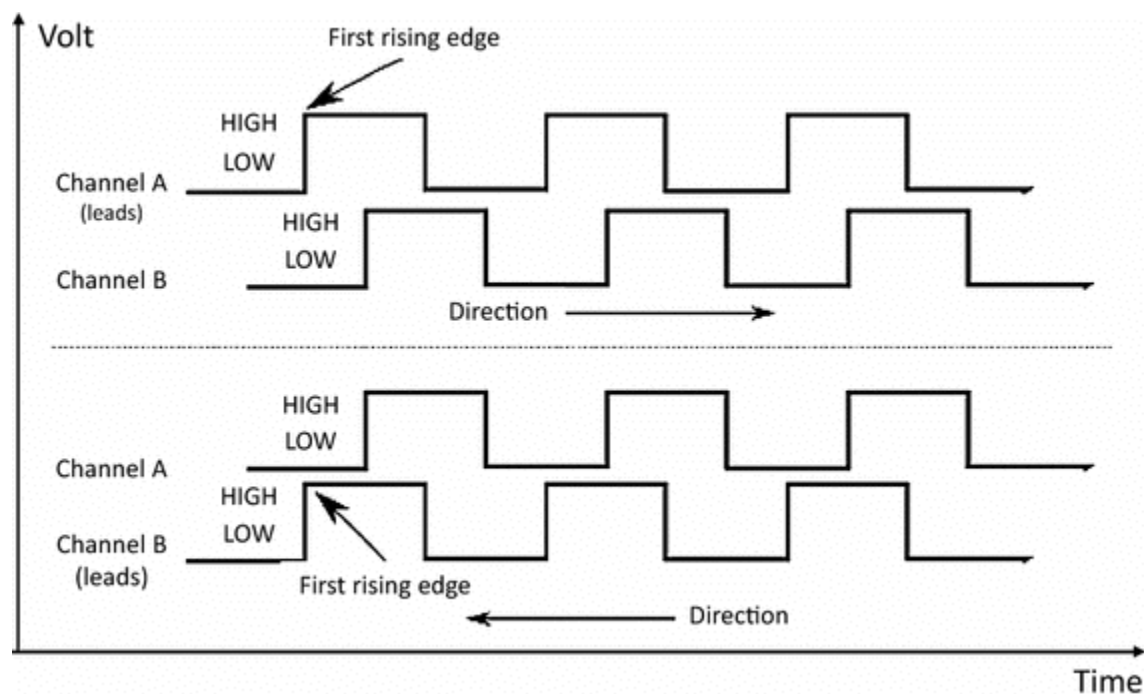
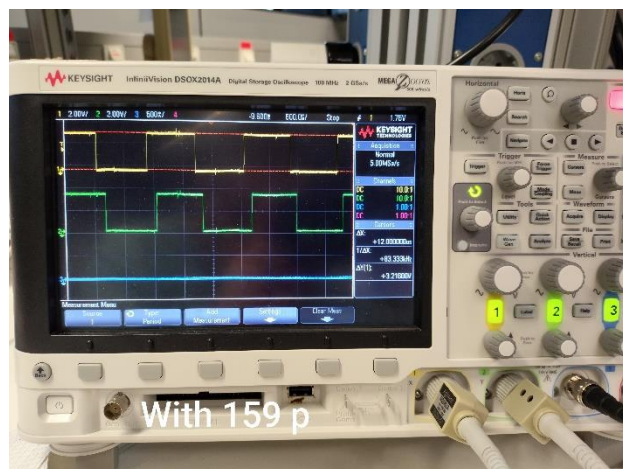


Figure 9



Figure 10



With 159 p

The encoders generate two quadrature signals that allow the firmware to determine rotation direction and calculate the motor angle by counting pulses. Besides implementing the PID controller, reliably processing these two signals proved to be the most difficult part of the project, both in hardware and software.

The issue first appeared as inconsistent and non-repeatable motor motion when executing the same commands. To isolate the problem, development started with only one motor enabled while closely observing the encoder interrupt routine and signal waveforms on an oscilloscope. At first, the signals looked correct.

A key observation was that motor behavior improved whenever oscilloscope probes—especially in $\times 10$ mode—were connected to the encoder signals. Although accuracy was still limited, the improvement indicated that the probes themselves were influencing the signals. This pointed to a signal-conditioning problem, as the probes add resistance and capacitance and therefore act as a low-pass filter.

To reproduce this effect, small capacitors were added intentionally to filter high-frequency noise. Different capacitor values were tested, but each change affected the pulse shape and required modifying the encoder count-per-revolution parameter in software. This showed that the encoder signals were sensitive to analog filtering and that the timing of the pulses was not stable.

The final solution was a carefully chosen RC filter, where the effective capacitance was the sum of several capacitors and parasitic effects. With a total capacitance of approximately 264 pF, the encoder signals became stable, pulse counting was consistent, and motor positioning became repeatable.

This experience highlights that proper signal conditioning is essential for reliable encoder-based position control, even when the digital logic appears correct.

$$C = 150 + 82 + 32 = 264 \text{ pf}$$

$$R = 100 \text{ ohm}$$

Figure 12

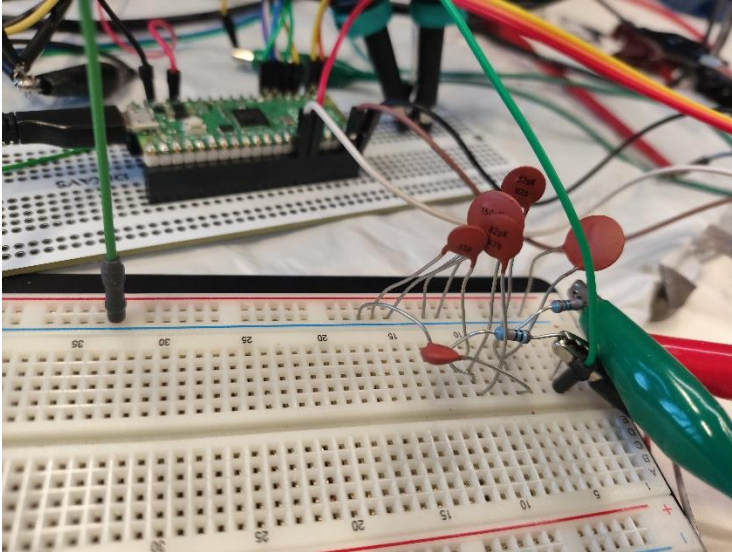
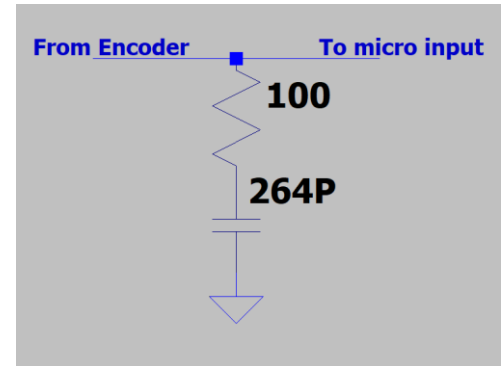


Figure 11



After applying the final RC filter, the motor behavior became stable and repeatable. However, when the second motor was added, a new problem appeared. The response of the second motor was noticeably different and not satisfactory.

This was surprising, since both motors used the same hardware, wiring concept, and software. A possible explanation was the longer wiring length (about one meter), which could introduce noise or signal reflections. Another unexpected observation was that changing the motion speed via command-line settings—for example, reducing it from 100% to 40% of the maximum—produced non-repeatable behavior. In theory, reducing speed should only lower the encoder signal frequency, not affect its shape or timing.

To investigate further, the encoder signals were examined more carefully using an oscilloscope. The captured waveforms show the A and B quadrature signals, along with the PWM signal. The signal amplitude was approximately 3.2 V, which appeared correct and therefore was initially not considered problematic. High-quality passive probes were used for both encoder channels during these measurements as you can see in provided pictures.

This indicated that the issue was not related to signal amplitude, but to more subtle timing or noise effects in the encoder signals.

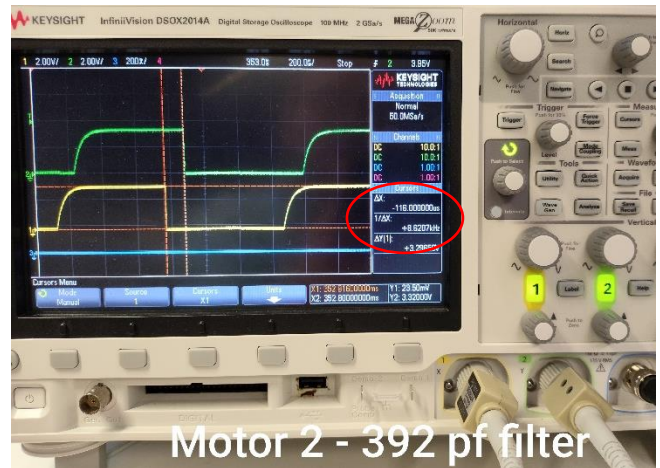
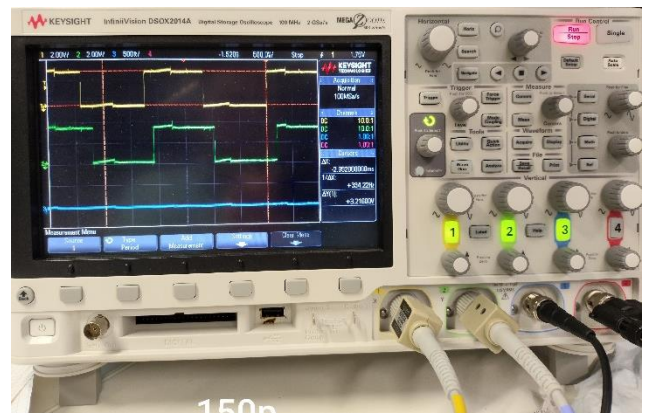
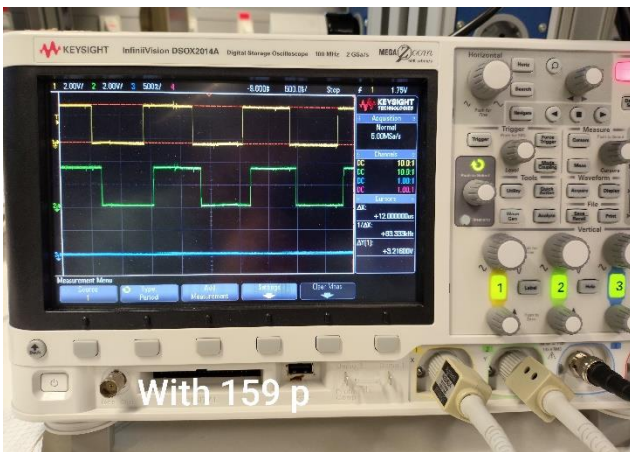
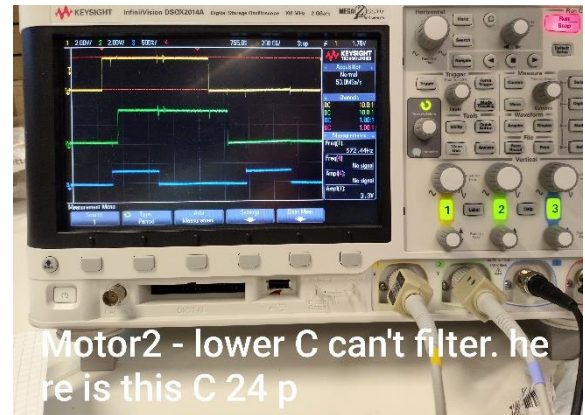
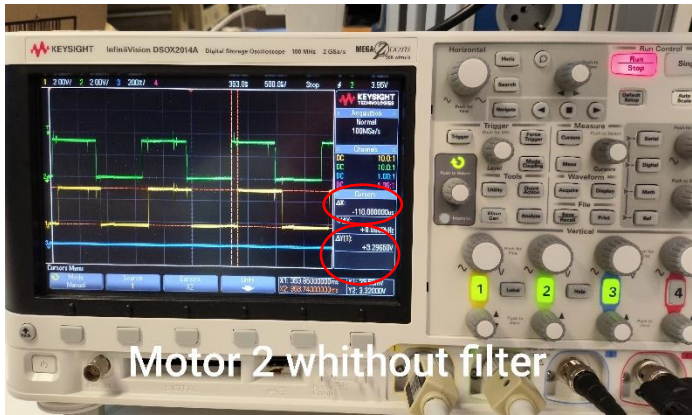


Figure 13

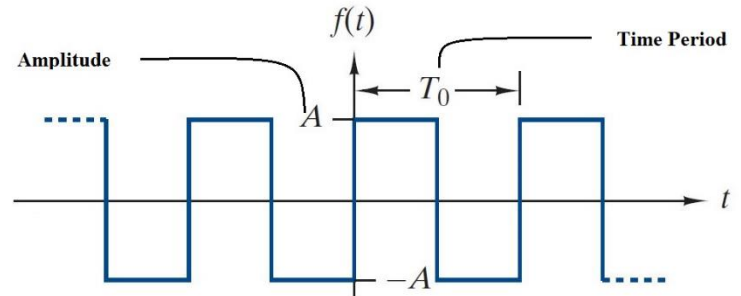
If noise is present but not easily visible on the oscilloscope, it still must be filtered. Even small disturbances can affect encoder edge detection and pulse counting.

Figure 14

From measurements at maximum motor speed, the encoder signal period was:

$$T_0 = 1.743\text{ms}$$

$$f = 1/T_0 = 573.6 \text{ Hz}$$



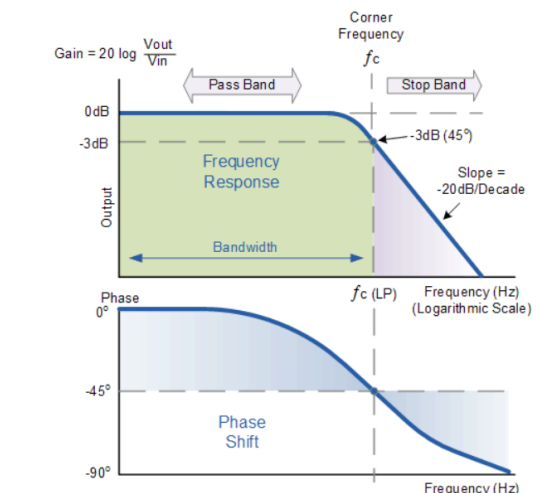
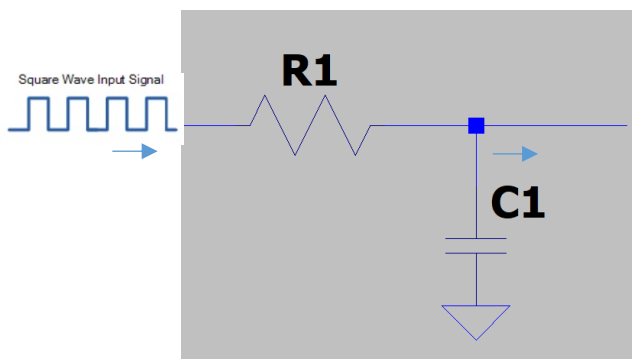
This frequency defines the minimum bandwidth required for reliable signal processing and was used as a reference when selecting the filter parameters.

To reliably detect signal edges, the cutoff frequency of the low-pass filter must be higher than the fundamental encoder frequency. Based on Fourier series considerations and to preserve sufficiently sharp edges, the cutoff frequency should be approximately 10–20 times the encoder frequency.

With $f=573.6 \text{ Hz}$, recommended cutoff range of about 6–12 kHz. A cutoff frequency of **10 kHz** was therefore selected.

Considering that $f_c = (2\pi RC)^{-1}$, with R 1 Kohm, C is 15 nF and with $R(100)$ $C(150\text{nF})$.

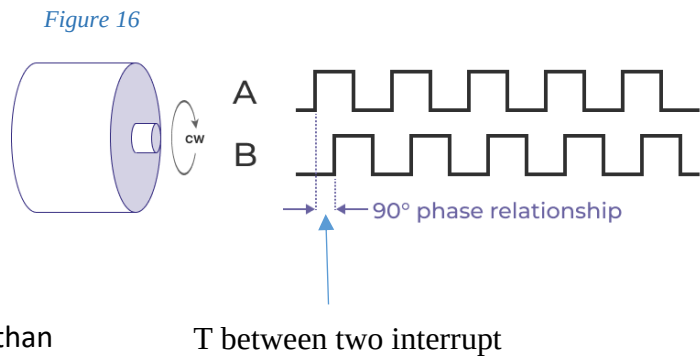
Figure 15



https://www.electronics-tutorials.ws/filter/filter_2.html

This filtering must be implemented in hardware when using an RC network. An RC filter with the calculated values was tested but it did not work! As I explained earlier, applying RC according Figure 11 was significantly improved the result and made the motor behavior repeatable. However, this solution still did not work reliably when the second motor was added. The reason for this issue is explained in the following sections of this document.

To increase resolution, the encoder ISR is triggered on both rising and falling edges rather than on signal level. With quadrature signals A and B, this results in four interrupts per encoder period. With a 90° phase shift between A and B, the maximum interrupt frequency becomes: $4 \times 573.6 \approx 2300$ Hz or in other words interrupts should not happen less than T-interrupt $\approx 435 \mu\text{s}$



This value provides a guideline for software filtering. If a software filter is used, its time threshold must be shorter than the expected interrupt interval. In this case, a filter time of approximately **400 μs** can be applied.

The software filter works as follows: if two consecutive interrupts occur within a time shorter than the filter threshold, the second interrupt is ignored and treated as noise. This simple timing-based approach effectively suppresses high-frequency noise while preserving valid encoder edges.

```
void motor::updateEncoder() {

#ifdef USE_ENCODER_FILTER // Filter: ignore too-fast interrupts
    unsigned long now = micros();
    unsigned long delta = now - lastEncoderMicros;

    if (delta < EncoderFilterMicroSecond) {
        return; // ignore noise
    }

    lastEncoderMicros = now; // update timestamp for next interrupt
#endif

    int MSB = digitalRead(encoderA_pin);
    int LSB = digitalRead(encoderB_pin);
    currState = (MSB << 1) | LSB;
    int sum = (prevState << 2) | currState;

    switch (sum) {
        case 0b1101: case 0b0100: case 0b0010: case 0b1011:
            EncoderCounter--;
    }
}
```

```

break;
case 0b1110: case 0b0111: case 0b0001: case 0b1000:
    EncoderCounter++;
    break;
}

prevState = currState;
}

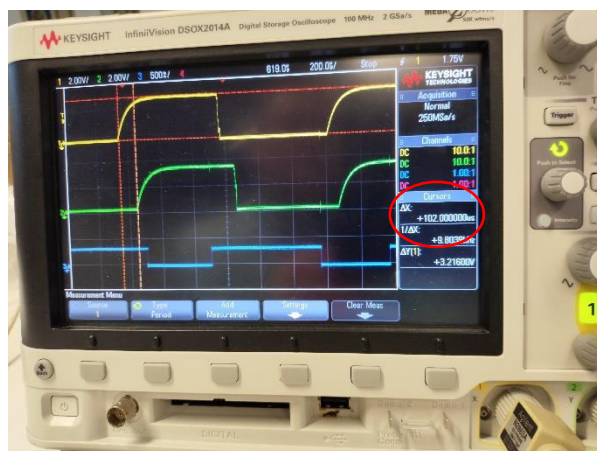
```

Encoder Signals in Practice

In reality, the encoder signals do not have an exact 90° phase shift between channels A and B. Moreover, this phase difference was not the same for the first and second motors. This can be seen clearly in the oscilloscope captures below.

The horizontal axis (Δx) is in microseconds (μs) and shows the time difference between the rising edges of channels A and B. As illustrated, the distance between corresponding rising edges differs between the two motors, even though the hardware and software setup is identical.

This deviation from the ideal quadrature assumption explains why identical filtering and software settings did not produce identical behavior for both motors.



102 μs

Figure 18



Figure 17

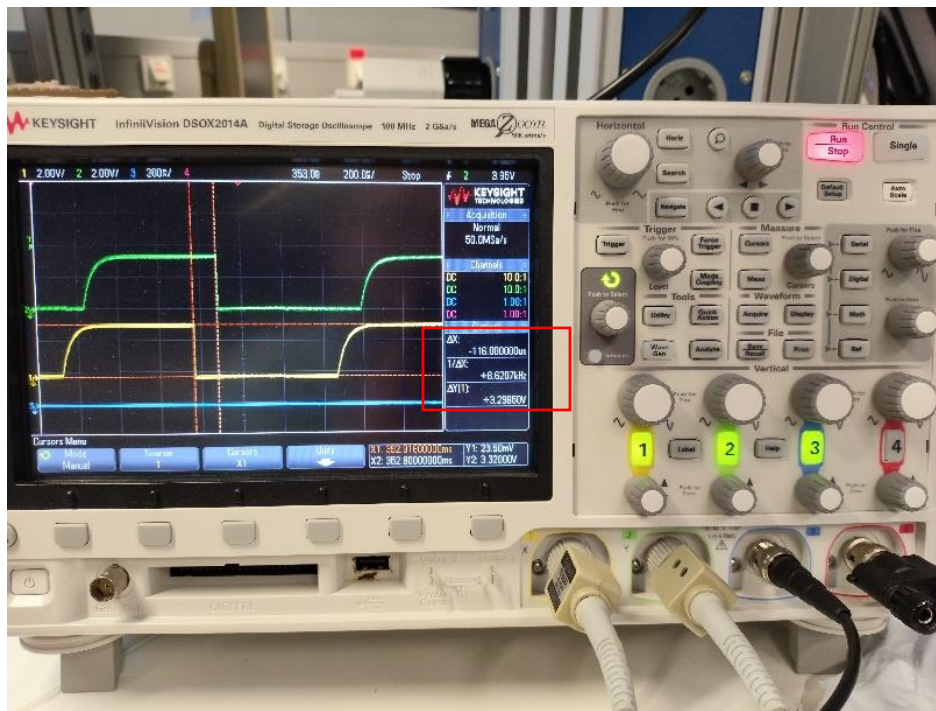


Figure 19

$$\Delta x = 116 \mu s$$

Therefore, based on the Δx values in the figure, the interval between two consecutive interrupts for the first motor (Motor 1) should be **less than 100 μs** .

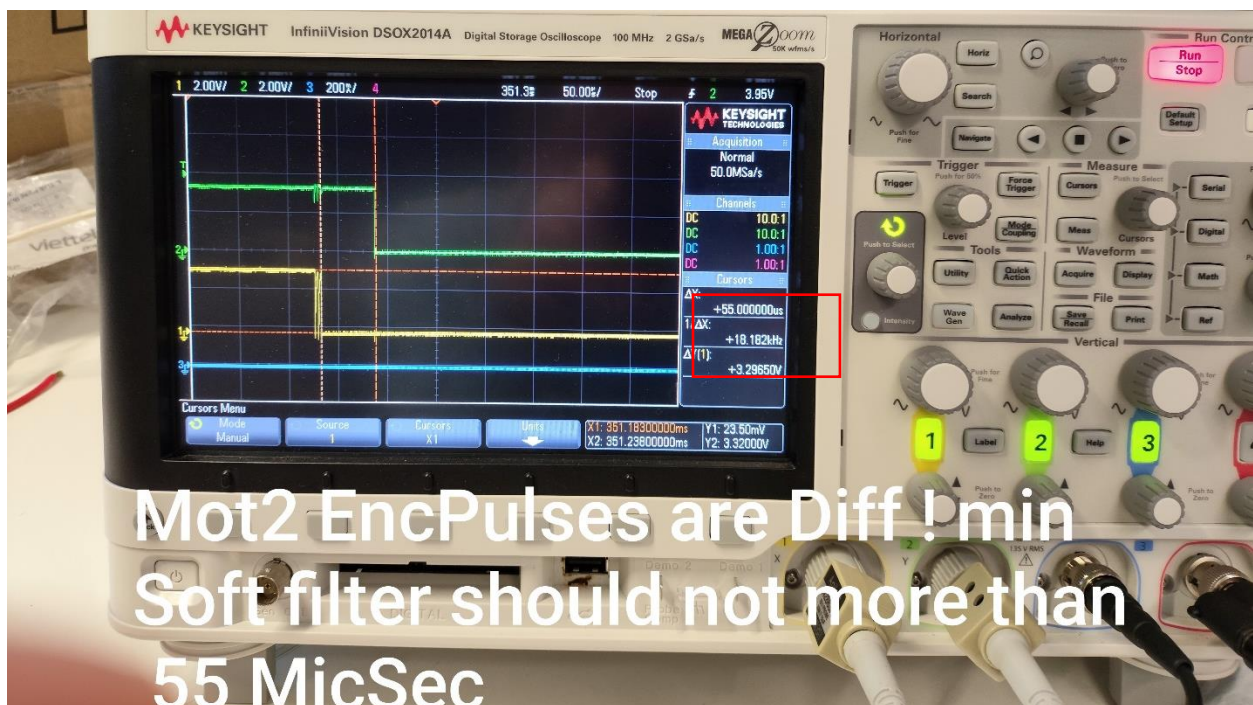


Figure 20

According to the figure, the filter for the second motor (tilt motor, Motor 2) should be **less than 50 μ s**. The chosen values were as follows:

```
/* ===== ENCODER FILTER ===== */  
constexpr unsigned ENCODER_FILTER_M1 = 50; //  $\mu$ s  
constexpr unsigned ENCODER_FILTER_M2 = 20;
```

Noise Captured ☺ :

When the motor approached the end of its interpolation, a large amount of noise was captured on the encoder signals.



Figure 21

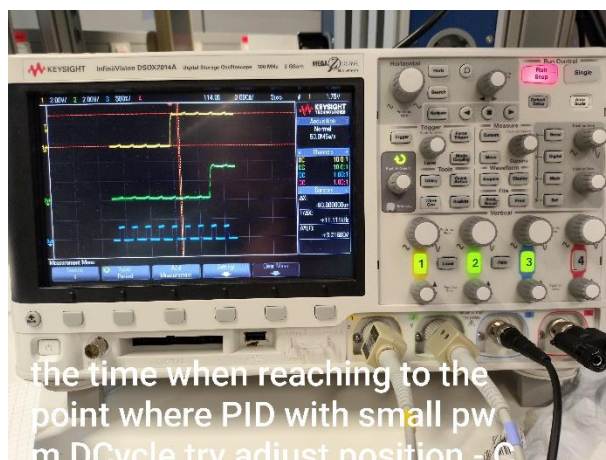
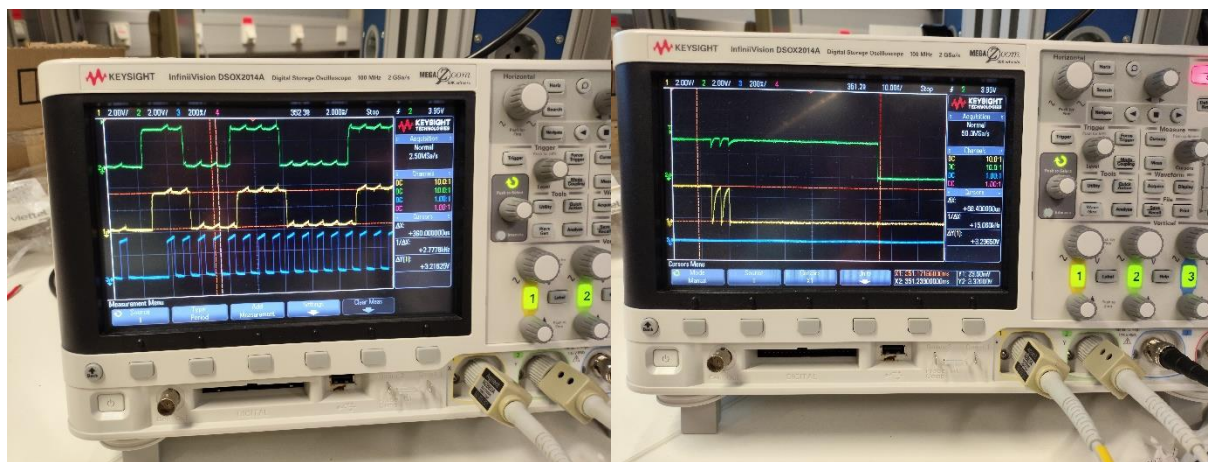


Figure 22

After applying the software filter, motor behavior became more repeatable. Later, I revisited the hardware filter, testing simple capacitors without a resistor. Capacitors ranging from 15 pF

to 1 nF (15, 33, 55, 67, 82, 89, 100, 150, 392, 1000 pF) were tried. The best result for both motors was **392 pF**.

The system is very sensitive to this value; deviations of more than ± 50 pF prevent proper operation. This optimal value could only be found through trial and error, as theoretical calculation was not straightforward. Previously, I had used different filters for each motor.

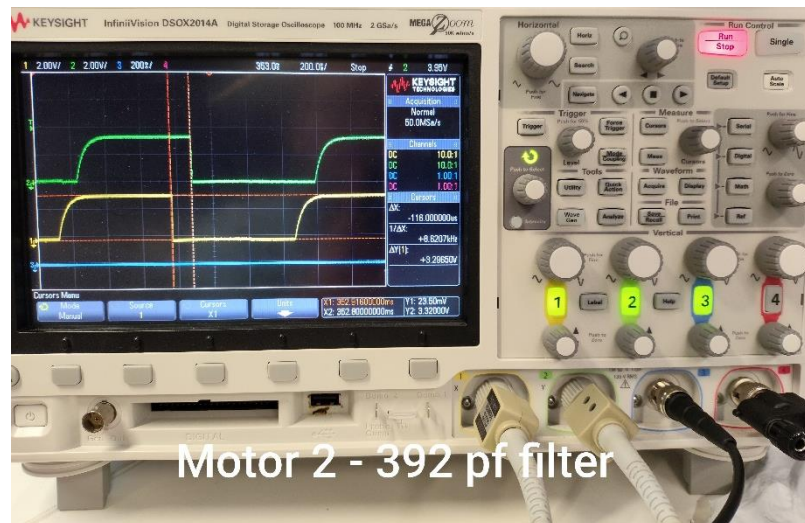
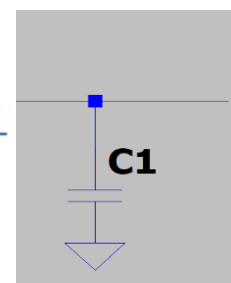


Figure 23

Figure 23

Thus, the final hardware filter consists of a single **392 pF capacitor**.

 **Hint:** A Schmitt trigger (e.g., 74HC14) can be used to clean encoder signals and reduce noise. For 3.3 V systems, the 74HCT14 is not suitable; use **74HC14**, which supports 3.3 V operation.



3.4 Zero Switch Signals

Zero switches are simple digital sensors that change state when an axis reaches a defined mechanical reference position. In this project, only two micro switches were available: one normally open (NO) and one normally closed (NC).

For safety, both switches should ideally be NC. In normal operation, an NC switch is closed, allowing the controller to monitor the loop signal (either Vcc or GND). If the wire is cut or disconnected, the loop is broken, and the controller interprets this as a triggered switch, ensuring a fail-safe condition. [Figure 7](#)

This principle is similar to level control in tanks or reservoirs, where high-high (HH) and low-low (LL) alarm switches are NC. A wire break will trigger the alarm even if the switch itself is not physically activated.

3.5 Emergency push button



Figure 24

In this project, no physical emergency push button was available, so wiring was used to simulate it.

- **Configuration:** The simulated button is normally closed (NC). In **normal operation**, the input is connected to **GND**, indicating safe operation.
- **Safety Principle:** A low signal (0 V) tells the controller that the system is operating normally.
- **Emergency Scenario:** When the button is pressed or a fault occurs, the contact opens, and the input is pulled **high** (Vcc, 3.3 V for the Pico), triggering an emergency stop.
- **Fail-Safe:** If the wire breaks or the button fails, the input rises to high, automatically stopping the system.

 **Hint:** In some industrial applications, both sides of the loop are monitored to detect wiring faults, with pull-ups and pull-downs used to ensure reliable fail-safe behavior.

4. Software Architecture

The software is written in C++ using both the Arduino IDE and, more recently, PlatformIO to gain experience with multiple development environments. Most of the work was done in **Arduino IDE 2.3.7** (upgraded from 2.3.6), which proved more reliable in my experience. For example, PlatformIO sometimes failed to detect certain code errors, while Arduino IDE handled them correctly.

PlatformIO offers advantages when used with VS Code, including AI-assisted editing, automatic comments, code archiving, and faster development. I have not yet explored the free Arduino AI assistant, but it may offer similar benefits:

<https://blog.arduino.cc/2025/04/17/code-faster-with-the-new-arduino-ai-assistant/>

The code is organized into several source files to maintain clear responsibilities and simplify maintenance. In PlatformIO, header files are stored separately from .cpp or .ino files, and the platformio.ini file is generated automatically. This file sometimes requires manual editing—for example, to include external libraries such as the PID library, which must be installed manually in Arduino IDE.

```
; PlatformIO Project Configuration File
;
; Build options: build flags, source filter
; Upload options: custom upload port, speed and extra flags
; Library options: dependencies, extra library storages
; Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:pico]
platform = https://github.com/maxgerhardt/platform-raspberrypi.git
board = pico
framework = arduino
board_build.core = earlephilhower
;upload_protocol = picotool //use for command line entry
upload_port = COM5
monitor_port = COM5
monitor_speed = 115200
monitor_filters = send_on_enter
lib_deps =                // PID library
    br3ttb/PID@^1.2.1
monitor_echo = yes ; added recently to enable echoing of typed characters
;otherwise when typing in the serial monitor, the characters are invisible
```

AI tools can assist with code changes and editing when needed. Another issue encountered with PlatformIO was that the serial monitor was less reliable than in the Arduino IDE. Some changes in setting should be done manually in platformio.ini.

Project's structure in platformIO:

```
DcServoControl/
├── include/
│   ├── motor.h
│   ├── motor_config.h
│   └── PID_vel.h
```

<https://github.com/peyman74/RaspberryPico-TwoArms-Robot-Positioning>

```

|   |— zero_calibration.h    ← NEW
|   |— system_safety.h     ← NEW
|   |
|   |— src/
|       |— main.cpp         ← REFACTORED
|       |— motor.cpp
|       |— PID_vel.cpp
|       |— zero_calibration.cpp ← NEW
|       |— system_safety.cpp ← NEW

```

Corrected & slide-ready:

In the Arduino IDE, all source files are located in the **DcServoControl** directory, alongside the main **DcServoControl.ino** file, which corresponds to **main.cpp** in PlatformIO.

The most important files are:

- **src/main.cpp** – main program, system initialization, and main loop.
- **src/motor.cpp / include/motor.h** – motor class with state handling and integrated PID control logic.
- **src/zero_calibration.cpp / include/zero_calibration.h** – zero-position interpolation and calibration routines.
- **src/system_safety.cpp / include/system_safety.h** – emergency switch handling and watchdog logic.
- **include/motor_config.h** – configuration constants such as PID parameters and timing settings.

4.1 Main Program (main.cpp)

Simplified:

```

void setup() {

    delay(2000); // Allow USB enumeration
    Serial.begin(115200);
    while (!Serial && millis() < 3000);
    /* ----- EMERGENCY INPUT ----- */
    pinMode(EMERGENCY_PIN, INPUT_PULLUP);

    DBGL("System initializing...");

    /* ----- CREATE MOTORS ----- */
    motors[0] = new motor(0, 10, 14, 15, 18, 19, 26, 27, KP, Ki_Ti, Kd_Td, PID_SAMPLE_TIME,
    ENCODER_FILTER_M1);
}

```



```

motors[1] = new motor(1, 11, 16, 17, 20, 21, 26, 28, KP, Ki_Ti, Kd_Td, PID_SAMPLE_TIME,
ENCODER_FILTER_M2);

/* ----- INITIALIZE MOTORS ----- */
for (int i = 0; i < numMotors; i++) {
    motors[i]->begin();
}

/* ----- ZERO INTERPOLATION ----- */
// Skip zero calibration if no zero switches installed
zeroInterpolationDone = !zeroSwitchInstalled;
watchdog_enable(HW_WATCHDOG_MS, false); // hardware watchdog activated

DBGL("System ready.");
}

void loop() {
    /* ----- WATCHDOG MONITORING ----- */
    watchdog_update();
    lastLoopKick = millis(); // Update timestamp for software watchdog

    /* ----- EMERGENCY MONITORING ----- */
    checkAndHandleEmergency();
    if (emergencyTriggered) {
        softwareWatchdogCheck();
        return; // Skip normal operation during emergency
    }

    /* ----- ZERO CALIBRATION ----- */
    // Interpolation needed at startup AND after emergency (industrial requirement)
    if (!zeroInterpolationDone) {
        zeroSearchMotors(); // Blocking version

        return; // Skip normal operation until calibration complete
    }
    // During normal operation, zero switch acts as an emergency/limit switch
    if (zeroSwitchInstalled && zeroInterpolationDone)
    {
        bool anyCollision = false;

        for (int i = 0; i < numMotors; i++) {
            if (!motors[i]->getZeroSwitchPinState())
            {

```

```

    anyCollision = true;
    motors[i]->forceStop();
}
}

if (anyCollision) {
    zeroInterpolationDone = false;
// Require re-zero interpolation after collision
    return;    // Go back to start of loop, skip Serial/controls
}
}

/* ----- READ SERIAL COMMAND AND PARSING ----- */
if (Serial.available() > 0) {

    String input = Serial.readStringUntil('\n');
    ...
    // Parse command: "P 30 60" or "S 30 60" or "M"
    ...

    /* --- MANUAL MODE COMMAND --- */
    /* --- MOVEMENT COMMANDS (P/S + SETPOINTS) --- */
    motors[i]->setSetpoint(constrain(tokens[i+1].toDouble(), 0, 360));

    // Check if already at target position

    // Print motor configuration and status

    /* ----- CONTROL LOGIC ----- */
    switch (motors[0]->getMode()) {
        case operationMode::MANUAL_MODE: // develop the code
            ...
            break;
        case operationMode::SEQUENTIAL_MOVEMENT:
            ...
            break;
        case operationMode::PARALLEL_MOVEMENT:
            // Process all motors simultaneously
            for (int i = 0; i < numMotors; i++)
            {
                if (fabs(motors[i]->getError()) > MAX_ACCEPABLE_ERR && !motors[i]->
getInterpolationStatus()) {
                    // Still moving to target
                    motors[i]->getPID()->Compute();

```

```

    } else {
        // Target reached
        motors[i]->setOutput(0);
        motors[i]->setInterpolationStatus(true);
        motors[i]->setStatus(MotorStatus::STOP_REST);
    }

    motors[i]->setMotor(); //put the out val-calculated in PID into Pico output
}
break;
/* ----- SOFTWARE WATCHDOG (AFTER CONTROL) ----- */
softwareWatchdogCheck();
}

// The end

```

The main program performs:

- Hardware initialization (serial, emergency pin, motors).
- Creation and initialization of motor objects.
- Zero interpolation at startup if zero switches are installed.
- The main loop, which repeatedly:
 1. Updates the watchdog timestamp.
 2. Checks the emergency input.
 3. Runs zero interpolation if required.
 4. Checks zero-switch collisions during normal operation.

I should add that I tried to use zero switch in two main rolls. Zero switch and limit/emergency switch. If the zero interpolation don't need (during normal interpolation) then its roll change to emergency/limit switch. In industrial application they should be separated and for each end of an axis we have a limit switch. Again that is always normally close for fail safe.
 5. Reads and parses serial commands.
 6. Executes the control logic for the active operating mode.
 7. Calls the software watchdog check.

4.2 Motor Abstraction (motor.*)

Each motor is represented by a `motor` class instance. This class encapsulates:

- Encoder reading and position calculation.
- PID controller object and tuning parameters.
- Current setpoint and error.
- Operating mode (manual, parallel, sequential).
- Status (RUN, STOP_REST, etc.).

The class also provides helper functions such as:

- Initializing the motor for manual or automatic control.
- Applying the PID output to the motor driver.
- Stopping the motor immediately (``forceStop``) when required by safety logic.

4.3 Zero Interpolation (`zero_calibration.*`)

Zero interpolation is used to find an accurate reference position for each.

At startup, and after certain safety events like emergency or reset, the system:

- Moves the motor with lower speed towards the zero switch.
- Detects the exact point where the zero switch becomes active.
- move a little bit in reverse to release zero switch.
- Reset the encoder count to zero at that moment and defines it as angle 0°.

This allows all subsequent angles (0° to 360°) to be measured and controlled relative to a well-defined mechanical reference. It is obvious that we can limit any axis movement like (0° to 270°) in the software.

4.4 Safety and Watchdog Logic (`system_safety.*`)

The safety module provides two main functions:

- ``checkAndHandleEmergency()`` – monitors the emergency input, stops all motors, and manages the transition back to normal operation.
- ``softwareWatchdogCheck()`` – detects if any loop in the program including main loop or PID interpolation loop in different mode has stalled and, if so, requests a system reset using a short hardware watchdog timeout.

These functions are called regularly from ``loop()`` in ``main.cpp`` to ensure that safety is always active.

Control and Operating Modes

5.1 Modes

The system supports three main modes, selectable via serial commands:

1. Manual Mode (M or m)

- Both motors are prepared for manual control.
- The user can move the motors directly, depending on the implementation in the motor class.

Hint: In this project, a potentiometer was used to provide setpoint commands. In industry, a “hand wheel” is more common, which usually includes an encoder rather than a potentiometer.

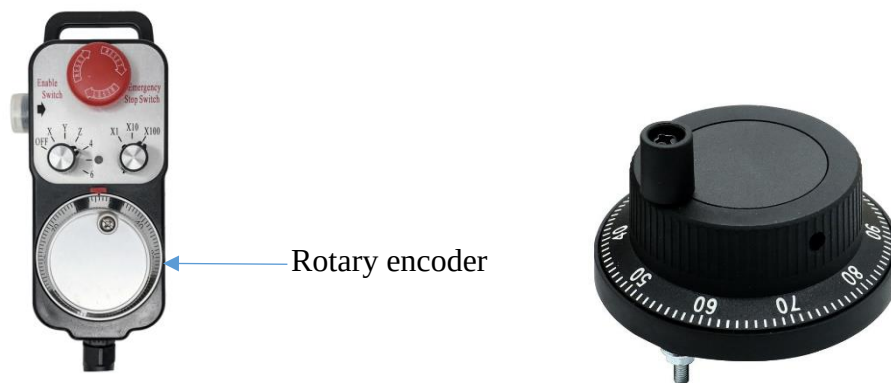


Figure 25

The hand-wheel setup includes a rotary encoder, an emergency push button, and two selectable knobs: one for choosing the axis to move and the other for setting the movement speed (feed).

2. Parallel Movement Mode (P or p)

- Command syntax: P angle1 angle2
- Both motors move simultaneously toward their target angles.

- The main loop updates both motors on every it

3. Sequential Movement Mode (S or s)

- Command syntax: S angle1 angle2
- Motor 1 moves to its target first. Motor 2 only starts moving after Motor 1 reaches its position.
- This behavior is implemented in the SEQUENTIAL_MOVEMENT case of the main control switch.

5.2 PID Position Control

Each motor uses a PID controller to reach and maintain its target angle.

Basic concept:

- $\text{error} = \text{setpoint} - \text{position}$
- The PID controller calculates the control output using the error, its integral, and its derivative
- The output is sent to the motor via the motor driver using `motor::setMotor()`

As mentioned earlier, besides encoder signal conditioning, the other major challenge was PID control and its tuning. Two different methods were implemented, each with its own tuning approach:

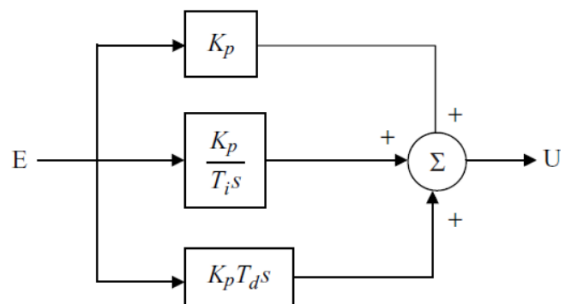
- **Velocity control** (implemented in `PID_vel.cpp`)
- **Classic position control**, including saturation, integral wind-up prevention, and derivative kick handling—important for accurate position control

Users can select the desired PID method in `motor_config.h`.

```
/* ===== PID TUNING (GLOBAL DEFAULTS) ===== */
#if USE_CLASSICAL_PID
constexpr double KP    = 5.0;
constexpr double Ki_Ti = 0.0; // Ki
constexpr double Kd_Td = 0.25; // Kd
constexpr int PID_SAMPLE_TIME = 10;
#else
constexpr double KP    = 0.9;
constexpr double Ki_Ti = 9000.0; // Ti
constexpr double Kd_Td = 0.0; // Td
```

```
constexpr int PID_SAMPLE_TIME = 100;
#endif
```

For this project, the proportional gain K_p is the most important parameter in both PID methods. The integral term is kept very small to avoid overshoot and oscillations—in the Classic PID, it is even set to zero. The derivative term in Classic PID is 0.25, calculated from the position measurement rather than the error, which helps to dampen movement. In velocity mode, the derivative term is zero.



Continuous-time system PID controller

$$u(t) = K_p e(t) + K_i \int_0^t e(t) dt + K_d \frac{de(t)}{dt} + u_0,$$

$$K_i = \frac{K_p}{T_i} \quad \text{and} \quad K_d = K_p T_d.$$

Equation 1

To implement the PID controller using a digital computer we have to convert this equation from a continuous to a discrete representation. There are several methods for doing this and the simplest is to use the trapezoidal approximation for the integral and the backward difference approximation for the derivative:

$$\frac{de(t)}{dt} \approx \frac{e(kT) - e(kT - T)}{T} \quad \text{and} \quad \int_0^t e(t) dt \approx \sum_{k=1}^n T e(kT).$$

Equation 2

$$u(kT) = K_p \left[e(kT) + T_d \frac{e(kT) - e(kT - T)}{T} + \frac{T}{T_i} \sum_{k=1}^n e(kT) \right] + u_0.$$

Velocity equation approach:

here K is K_p :

a) Right point integration:

$$y(t) = K \left(e + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_d \dot{e} \right)$$

$$u_k = K \left(e_k + \frac{T}{T_i} [e_1 + e_2 + e_3 + \dots + e_k] + T_d \cdot \frac{e_k - e_{k-1}}{T} \right)$$

$$u_{k-1} = K \left(e_{k-1} + \frac{T}{T_i} [e_1 + e_2 + e_3 + \dots + e_{k-1}] + T_d \cdot \frac{e_{k-1} - e_{k-2}}{T} \right)$$

Subtracting the two equations leads to the algorithm in standard form:

$$u_k - u_{k-1} = K \left(e_k - e_{k-1} + \frac{T}{T_i} [e_k] + T_d \cdot \frac{e_k - 2e_{k-1} + e_{k-2}}{T} \right)$$

or

$$u_k = u_{k-1} + \left(K + \frac{K}{T_i} T + K \frac{T_d}{T} \right) e_k + \left(-K - 2K \frac{T_d}{T} \right) e_{k-1} + K \left(\frac{T_d}{T} \right) e_{k-2}$$

Equation 3

b) Left point integration:

$$u_k - u_{k-1} = K \left(e_k - e_{k-1} + \frac{T}{T_i} [e_{k-1}] + T_d \cdot \frac{e_k - 2e_{k-1} + e_{k-2}}{T} \right)$$

or

$$u_k = u_{k-1} + \left(K + K \frac{T_d}{T} \right) e_k + \left(-K + \frac{K}{T_i} T - 2K \frac{T_d}{T} \right) e_{k-1} + K \left(\frac{T_d}{T} \right) e_{k-2}$$

Equation 4

c) Mid point integration:

$$u_k - u_{k-1} = K \left(e_k - e_{k-1} + \frac{T}{T_i} \left[\frac{e_k - e_{k-1}}{2} + e_{k-1} \right] + T_d \cdot \frac{e_k - 2e_{k-1} + e_{k-2}}{T} \right)$$

or

$$u_k = u_{k-1} + \left(K + K \frac{T_d}{T} + K \frac{T}{2T_i} \right) e_k + \left(-K + K \frac{T}{2T_i} - 2K \frac{T_d}{T} \right) e_{k-1} + K \left(\frac{T_d}{T} \right) e_{k-2}$$

Equation 5

a, b, and c may look different, but they are in fact similar. This form of PID is known as the **velocity PID controller**. In this approach, the current control action uses the previous control value as a reference. Because only changes in the control action are applied, the velocity PID provides smoother, bumpless control when the error is small. However, if the error is large, the response can be slow, especially when the integral time T_i is large.

According to D. Ibrahim in *Microcontroller Based Applied Digital Control* (Chapter 9.2), this method is effective for initial PID implementation. In this project, the first test with one motor for tuning was named **SpeedLoopKpTiTd**, which can be found at the following link:

<https://bwsyncandshare.kit.edu/apps/files/files/5241844567?dir=/ProjectB>

```
bool PID_Function() {
    unsigned long now = millis();
    if (now - last_time >= PID_SAMPLE_TIME) {

        current_position = getPosition();
        // kind of filtering
        current_position = 0.8 * current_position + 0.2 * getPosition();
        //current_position = (getPosition() + lastPos1 + lastPos2) / 3;
        error = setpoint - current_position;

        // PID midpoint integration
        output += constant1 * error + constant2 * error_1 + constant3 * error_2 ;
        error_2 = error_1;
        error_1 = error;
        output = constrain(output, -motorMaxSpeed, motorMaxSpeed);
        if (fabs(error) < 0.5)
            filterCount++;
        else
            filterCount = 0;

        if (filterCount >= BOUNCING) {
            interpolationDone = true;
            output = 0;
            filterCount = 0;
        }
        last_time = now;
    }
    return interpolationDone;
}

void setup() {
```

```

    ... hardware initializing functions
// Disable internal temperature sensor to avoid noise
adc_set_temp_sensor_enabled(false);
pwmInit();
prevState = (digitalRead(encoderA_pin) << 1) | digitalRead(encoderB_pin);
attachInterrupt(digitalPinToInterrupt(encoderA_pin), updateEncoder, CHANGE);
attachInterrupt(digitalPinToInterrupt(encoderB_pin), updateEncoder, CHANGE);

Serial.println("Enter commands: S <setpoint> <Kp> <Ti> <Td>");
}

void loop() {

// --- Serial Command Input ---
if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    cmd.toUpperCase();
    if (cmd.startsWith("S")) {
        Serial.print("\n Entered S");
        double newSetpoint, newKp, newTi, newTd;
        sscanf(cmd.c_str(), "S %lf %lf %lf %lf", &newSetpoint, &newKp, &newTi, &newTd);
        setpoint = constrain(newSetpoint, 0, 360);
        Kp = newKp;
        Ti = newTi;
        Td = newTd;
        // output += Kp * (1 + Td/T) * error + Kp * (T/Ti - 1 - 2*Td/T) * error_1 + Kp * Td/T * error_2
    ); or
        // output += Kp * (1 + Kd) * error + Kp * (Ki - 1 - 2*Kd) * error_1 + Kp * Kd * error_2 );
        constant1 = Kp * (1 + Td/pidSampleTime);
        if (Ti != 0) constant2 = Kp * (pidSampleTime/Ti - 1 - 2*Td/pidSampleTime);
        constant3 = Kp * Td/pidSampleTime;
        // output += constant1 * error + constant2 * error_1 + constant3 * error_2 ;
        interpolationDone = false;
        delay(2000);
    }
}

// --- Run PID loop ---
// if (!interpolationDone)
PID_Function();
setMotor(output);
}

```

After extensive tuning, the following PID values were selected. With these settings, the PID loop operates with minimal vibration, and the interpolation process is smooth and accurate. This program can serve as a good reference for anyone developing a velocity control-based system.

```
double pidSampleTime = 0.01; //10ms
double Kp = 6.0, Ti = 0.6, Td = 0, KP=0, KI=0, KD=0, constant1, constant2, constant3;
```

Next, I continued with Chapter 9.2 of D. Ibrahim's book, where he explained his approach to PID control. I decided to implement a second method, which I called the "Classic" PID, incorporating the ideas discussed in the book. To explain this method, let us first review the key points from the chapter.

"Saturation and Integral Wind-Up"

In practical applications, the output value of a control action is limited by physical constraints. For example, the maximum voltage output from a device is limited. Similarly, the maximum flow rate that a pump can supply is limited by the physical capacity of the pump.

As a result of this physical limitation, the error signal does not return to zero, and the integral term keeps adding up continuously. This effect is called **integral wind-up** (or integral saturation), and as a result, long periods of overshoot can occur in the plant response.

A simple example of what happens is the following: Suppose we wish to control the position of a motor and a large setpoint change occurs, resulting in a large error signal. The controller will then try to reduce the error between the setpoint and the output. The integral term will grow by summing the error signals at each sample, and a large control action will be applied to the motor. But because of the physical limitation of the motor electronics, the motor will not be able to respond linearly to the applied control signal. If the setpoint now changes in the other direction, then the integral term is still large and will not respond immediately to the setpoint request. Consequently, the system will have a poor response when it comes out of this condition.

The integral wind-up problem affects positional PID controllers. With velocity PID controllers, the error signals are not summed up, and as a result, integral wind-up will not occur, even though the control signal is physically constrained.

Many techniques have been developed to eliminate integral wind-up from PID controllers, and some of the popular ones are as follows:

- Stop the integral summation when saturation occurs. This is also called **conditional integration**. The idea is to set the integrator input to zero if the controller output is saturated and the input and output are of the same sign.
- Fix the limits of the integral term between a minimum and a maximum.

- Reduce the integrator input by some constant if the controller output is saturated. Usually, the integral value is decreased by an amount proportional to the difference between the unsaturated and saturated (i.e., maximum) controller output.
- Use the velocity form of the PID controller.

Derivative Kick

Another possible problem when using PID controllers is caused by the derivative action of the controller. This may happen when the setpoint changes sharply, causing the error signal to change suddenly. Under such conditions, the derivative term can give the output a sudden change, known as a **derivative kick**.

This is usually avoided in practice by moving the derivative term to the feedback loop. The proportional term may also cause a sudden kick in the output, and it is common to move the proportional term to the feedback loop."

Reference: Microcontroller Based Applied Digital Control / Dogan Ibrahim

To apply these considerations, I reviewed PID libraries available online. I tried a few, and the one below worked perfectly with my project. The flowchart of the applied PID is shown below.

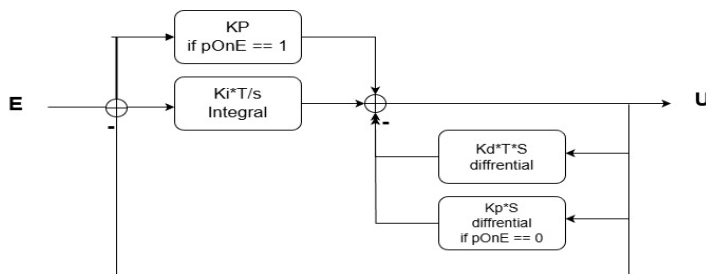
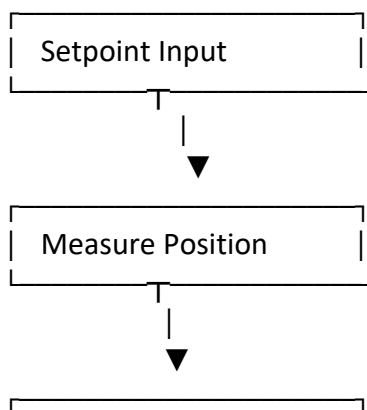
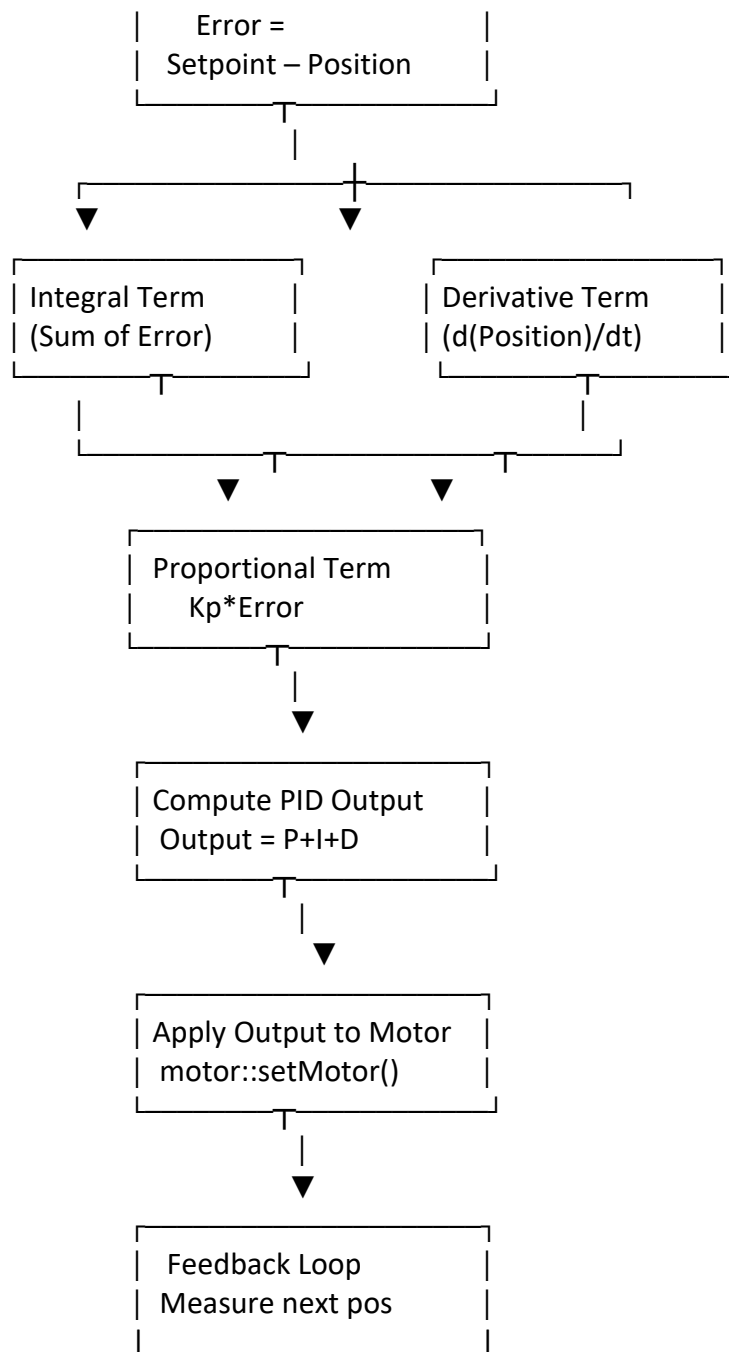


Figure 27

pOnE means proportional on error.





* Arduino PID Library - Version 1.1.1
 * by Brett Beauregard <br3ttb@gmail.com> brettbeauregard.com

```

bool PID::Compute()
{
  if(!inAuto) return false; // if it is not in AUTOMATIC mode it returns

```

```

unsigned long now = millis();
unsigned long timeChange = (now - lastTime);
if(timeChange>=SampleTime)
{
    /*Compute all the working error variables*/
    double input = *myInput; // here input is the feedback or position
    double error = *mySetpoint - input;
    double dInput = (input - lastInput); // this is important part
    outputSum+= (ki * error);

    /*Add Proportional on Measurement, if P_ON_M is specified*/
    if(!pOnE) outputSum-= kp * dInput; /*Derivative Kick consideration*/

    if(outputSum > outMax) outputSum= outMax;
    else if(outputSum < outMin) outputSum= outMin;
    /* Fix the limits of the integral term between a minimum and a maximum*/
    /* preventing integral win up */
    /*Add Proportional on Error, if P_ON_E is specified*/
    double output;
    if(pOnE) output = kp * error;
    else output = 0;

    /*Compute Rest of PID Output*/
    output += outputSum - kd * dInput; /*Derivative Kick consideration*/
    /* as we see kd is always in the feedback loop */
    if(output > outMax) output = outMax;
    else if(output < outMin) output = outMin;
    *myOutput = output;

    /*Remember some variables for next time*/
    lastInput = input;
    lastTime = now;
    return true;
}
else return false;
}

```

Additionally, I tested *version 1.2.3* by *David Forrest and Brett Beauregard*. The following code was included to prevent saturation:

It Reduces the integrator input some constant if the controller output is saturated.

Codes added to previous version in *version 1.2.3*:

```

if(output > outMax){

```

```

    outputSum -= output - outMax; // backcalculate integral to feasibility
    output = outMax;
}
else if(output < outMin) {
    outputSum += outMin - output; // backcalculate integral to feasibility
    output = outMin;
}

```

But in my project, it wasn't effective, so I used the previous version.
In this PID library, T is already considered in the program:

```

double SampleTimeInSec = ((double)SampleTime)/1000;
kp = Kp;
ki = Ki * SampleTimeInSec; //new Ki
kd = Kd / SampleTimeInSec; //new kd

```

When this method is compared with the velocity-form PID equation, the Ki input is equivalent to $1/T_i$, which is then multiplied by the sampling time T inside the algorithm. Similarly, Kd corresponds to T_d , which is divided by T internally.

Thus, in this library (classical form), the effective gains are:

- $K_i = (1 / T_i) \cdot T$
- $K_d = T_d / T$

Therefore, when using this library, the user effectively enters $1/T_i$ and T_d , whereas in the velocity-form PID, the input parameters are directly T_i and T_d .

During tuning process, the following optimal parameters were identified:

```

#if USE_CLASSICAL_PID
constexpr double KP    = 5.0;
constexpr double Ki_Ti = 0.0; // Ki
constexpr double Kd_Td = 0.25; // Kd
constexpr int PID_SAMPLE_TIME = 10;

```

6. Safety Concept

Safety is a central part of this project. Three mechanisms are used:

<https://github.com/peyman74/RaspberryPico-TwoArms-Robot-Positioning>

- Emergency switch with fail-safe wiring.
- Zero switch used as a collision/limit switch during normal operation.
- Combined hardware and software watchdog.

6.1 Emergency push button

The emergency push button is connected to the `EMERGENCY\_PIN` with an external pull-up resistor:

- Normal condition: the button connects the input to ground → the pin reads LOW.
- Emergency condition or wire break: the input is pulled HIGH through the resistor.

The logic is intentionally inverted compared to some simple examples: HIGH means emergency. This means that even if the wire is cut or unplugged, the system safely enters the emergency state.

 **Hint:** The emergency push button is currently checked using a polling method. Initially, both interrupt and polling were implemented, but due to technical issues—requiring a soft restart of the Pico each time to upload updated code—the interrupt was removed. However, the correct approach is to use both methods together.

The interrupt service routine should be very short and only set a flag. The following parts of the code need to be modified or added:

```
volatile bool emergencyTriggered = false;

void emergencyISR() { /* interrupt with short body */
    emergencyTriggered = true;
}

void setup() {
    // Emergency stop input (internal pull-up). Active HIGH (NC switch to GND).
    // It is always high except when the "switch pressed" or "wires break".
    pinMode(EMERGENCY_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(EMERGENCY_PIN), emergencyISR, RISING);
    ...
}
```

```
// Check emergency pin and handle immediate-safe stop and reinitialization.
void checkAndHandleEmergency() {
```



```
// Detect emergency (interrupt OR polling)
if (!emergencyTriggered && digitalRead(EMERGENCY_PIN) == HIGH) {
    emergencyTriggered = true;
}

// FIRST entry into emergency
if (emergencyTriggered && !emergencyHandlingDone) { ...
```

Compare this with the current function in the `system_safety.cpp` file:

```
void checkAndHandleEmergency() {

    // Check emergency input (HIGH = emergency) - polling method
    if (digitalRead(EMERGENCY_PIN) == HIGH) {
        emergencyTriggered = true;
    }
}
```

In `checkAndHandleEmergency()`:

- If the pin reads HIGH, `emergencyTriggered` is set and all motors are immediately stopped. The PID controllers are switched to manual mode so they no longer drive the outputs.
- When the emergency condition is cleared (the pin goes LOW again), the function:
 - Prints a message to the serial terminal.
 - Re-enables the PID controllers.
 - Clears the emergency flags.
 - Sets `zeroInterpolationDone = false`, requiring a new zero interpolation before normal motion resumes.

6.2 Zero Switch as Collision / Limit Switch

At startup, the zero switches are used purely for finding the reference position. However, during normal operation, the zero switches are reinterpreted as collision or limit switches.

In the main loop, after zero interpolation has been completed, the program checks:

- If any zero switch becomes active (motor enters the "forbidden" zone).

If this happens:

- The affected motor is immediately stopped using `forceStop()`.
- A clear message is printed once to the serial terminal indicating which motor has collided and that the motor should be pulled out of the emergency zone.

- The flag ``zeroHandlingZoneDone`` is used to avoid printing the same message repeatedly in every loop iteration.
- ``zeroInterpolationDone`` is set to ``false`` to force a new zero interpolation after the collision has been cleared.
- The function returns early, so the rest of the loop (including serial command processing) is skipped while the collision persists.

6.3 Watchdogs

Two watchdog mechanisms are used:

1. Hardware watchdog

- Provided by the microcontroller.
- If not regularly updated, it resets the entire system.

2. Software watchdog

- The variable ``lastLoopKick`` is updated at the start of every ``loop()`` call.
- ``softwareWatchdogCheck()`` compares the current time with ``lastLoopKick``.
- If the difference exceeds a defined limit, the function logs a warning, enables a short hardware watchdog timeout, and waits for the reset.

By placing the software watchdog check at the end of the main loop, the program ensures that a successful loop iteration has been completed before the system is considered healthy.

7. User Interface and Operation

The user interacts with the system through a serial terminal (for example, the Serial Monitor in the IDE or an external terminal program).

The baud rate is 115200 baud.

7.1 Command Syntax

The main commands are:

- ``M`` or ``m``
Activate manual mode for all motors.
- ``P angle1 angle2``
Parallel movement: both motors move to the given angles.

- `S angle1 angle2`

Sequential movement: Motor 1 moves first, then Motor 2.

Angles are typically in the range 0° to 360°. The firmware constrains setpoints to this valid range.

7.2 Typical Operating Procedure

1. Power on the system.
2. Wait for the serial message indicating that the system is ready and zero interpolation is complete.
3. Send movement commands (P or S) with desired target angles.
4. Observe the serial output for information about setpoints, current positions, and PID parameters.
5. If an emergency or collision occurs:
 - The system stops motion and prints a message.
 - After resolving the cause (releasing the emergency switch or pulling the motor out of the collision zone), the system will require a new zero interpolation before further motion.

8. Conclusion and Future Work

In this project, a two-axis DC servo control system was designed and implemented using a Raspberry Pi Pico. The system can move two motors to commanded angles with encoder feedback and PID control. It supports manual, parallel, and sequential operating modes.

A strong focus was placed on safety. The emergency switch is wired in a fail-safe way so that a broken wire is treated as an emergency. Zero switches are used not only for initial calibration but also as collision or limit switches during normal operation. Hardware and software watchdogs monitor the health of the main loop and reset the system in case of a stall.

The tests carried out showed that the system behaves as expected and remains stable under different operating conditions, including power cycling and emergency events.

Possible future improvements include:

- More advanced PID tuning or automatic tuning methods.
- Add Bouncing for reduce vibration in velocity PID method.
- A graphical user interface on the PC for easier control and visualization.
- “Backlash” effects could be considered and compensated to improve system accuracy.

9. Appendices

Software code:

You can find up-to-dated code in this link:

<https://bwsyncandshare.kit.edu/apps/files/files/5241844567?dir=/ProjectB>

Arduino IDE formats:

RobatWithDCMotorContrl/

- motor.h
- motor_config.h
- PID_vel.h
- zero_calibration.h
- system_safety.h
- RobatWithDCMotorContrl.cpp
- motor.cpp
- PID_vel.cpp
- zero_calibration.cpp
- system_safety.cpp

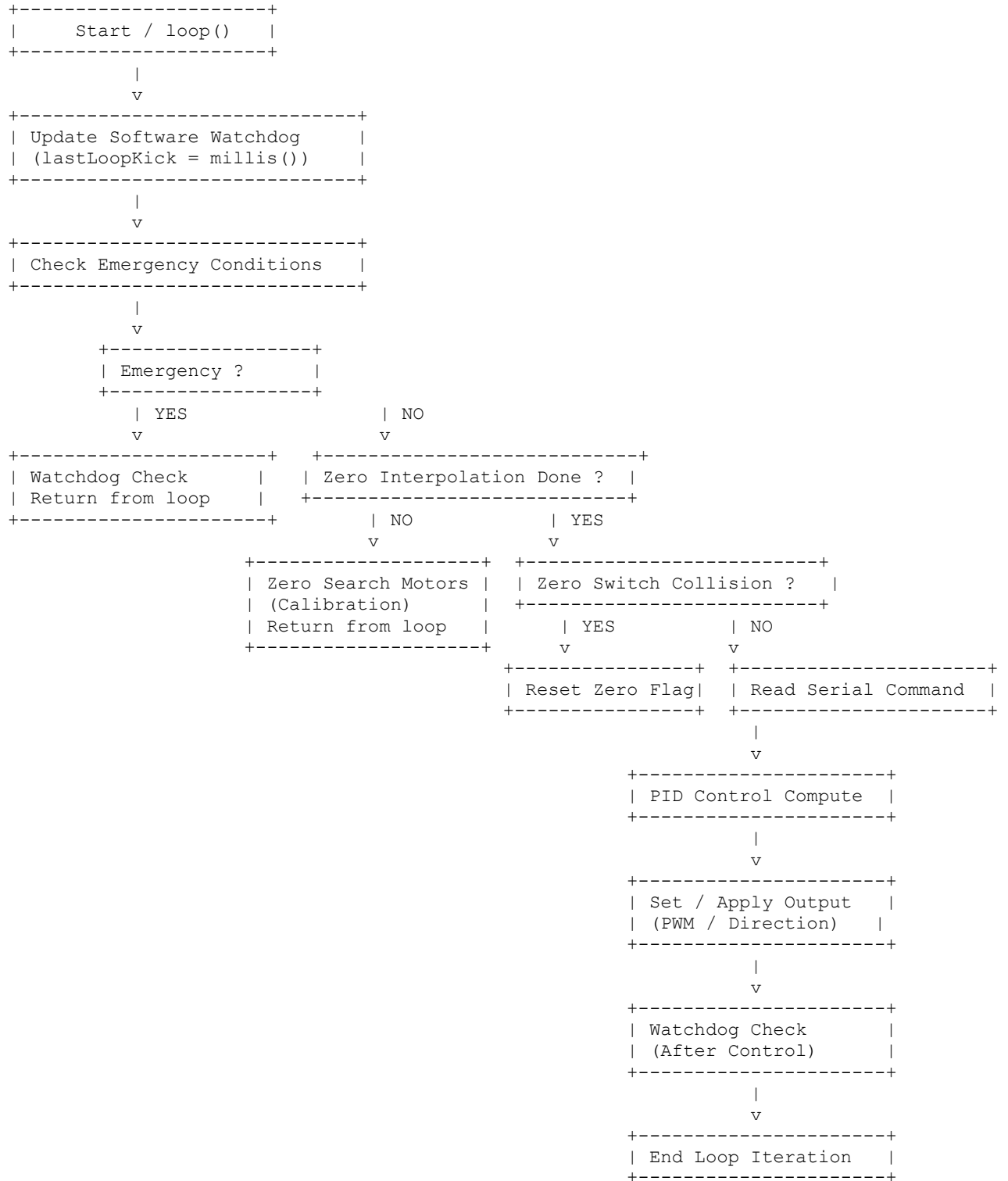
Project's structure in platformIO:

DcServoControl/

- include/
 - motor.h
 - motor_config.h
 - PID_vel.h
 - zero_calibration.h
 - system_safety.h
- src/
 - main.cpp
 - motor.cpp
 - PID_vel.cpp
 - zero_calibration.cpp
 - system_safety.cpp

RobatWithDCMotorContrl.ino or main.cpp code is printed here. Please refer to the provided link to access whole program.

The algorithm is like below:




```

/**
 * @file RobatWithDCMotorContrl.ino
 * @brief DC Servo Control System - Main Program
 */
#include "motor.h"
#include "motor_config.h"
#include "zero_calibration.h" // Zero position calibration functions
#include "system_safety.h"    // Emergency and watchdog safety systems

/* ===== GLOBAL MOTOR CONFIGURATION ===== */
int numMotors = 2; // Number of motors in system
motor** motors = new motor*[numMotors];

/* ===== SYSTEM STATE FLAGS ===== */
bool zeroInterpolationDone = false;
bool emergencyTriggered = false;
bool emergencyHandlingDone = false;
bool zeroHandlingZoneDone = false;
bool zeroSwitchInstalled = true; // Set to true if any motor has zero switch

/* ===== TIMING ===== */
unsigned long lastLoopKick = 0; // Software watchdog timestamp

/* ===== HARDWARE PINS ===== */
constexpr uint8_t EMERGENCY_PIN = 4; // Emergency stop input (external pull-up)
constexpr uint8_t BOOTSEL_PIN = 6;   // Wire to GND to force bootloader

void setup() {
    delay(2000); // Allow USB enumeration

#ifdef PICO_DEFAULT_LED_PIN
    pinMode(PICO_DEFAULT_LED_PIN, OUTPUT);
    digitalWrite(PICO_DEFAULT_LED_PIN, HIGH);
    delay(300);
    digitalWrite(PICO_DEFAULT_LED_PIN, LOW);
#endif

    Serial.begin(115200);
    while (!Serial && millis() < 3000);

    /* ----- FORCE BOOTSEL (WIRE TO GND) ----- */
    pinMode(BOOTSEL_PIN, INPUT_PULLUP);

```

```

if (digitalRead(BOOTSEL_PIN) == LOW) {
    Serial.println("BOOTSEL forced");
    delay(50);
    reset_usb_boot(0, 0);
}

/* ----- EMERGENCY INPUT ----- */
pinMode(EMERGENCY_PIN, INPUT_PULLUP);

DBGL("System initializing...");

/* ----- CREATE MOTORS ----- */
/* motor(int id, int pwm_pin, int forward_pin, int backward_pin,
    int encoderA_pin, int encoderB_pin, int potentiometer_jogging_pin,
    int zero_switch_pin, double Kp, double Ki, double Kd, int pidSampleTime,
volatile unsigned EncoderFilterMicroSecond); */
motors[0] = new motor(0, 10, 14, 15, 18, 19, 26, 27, KP, Ki_Ti, Kd_Td,
PID_SAMPLE_TIME, ENCODER_FILTER_M1);
motors[1] = new motor(1, 11, 16, 17, 20, 21, 26, 28, KP, Ki_Ti, Kd_Td,
PID_SAMPLE_TIME, ENCODER_FILTER_M2);

/* ----- INITIALIZE MOTORS ----- */
for (int i = 0; i < numMotors; i++) {
    motors[i]->begin();
}

/* ----- ZERO INTERPOLATION ----- */
// Skip zero calibration if no zero switches installed
zeroInterpolationDone = !zeroSwitchInstalled;

Serial.setTimeout(50);
watchdog_enable(HW_WATCHDOG_MS, false); // Keep OFF during development

DBGL("System ready.");
}

void loop() {

    watchdog_update();
    lastLoopKick = millis(); // Update timestamp for software watchdog

    /* ----- EMERGENCY MONITORING ----- */
    checkAndHandleEmergency();
    if (emergencyTriggered) {

```

```

softwareWatchdogCheck();
return; // Skip normal operation during emergency
}

/* ----- ZERO CALIBRATION ----- */
// Interpolation needed at startup AND after emergency (industrial requirement)
if (!zeroInterpolationDone) {
    #if USE_ZERO_NON_BLOCKING
        zeroSearchMotorsNonBlocking(); // State machine version (recommended)
    #else
        zeroSearchMotors(); // Blocking version
    #endif
    return; // Skip normal operation until calibration complete
}
// During normal operation, zero switch acts as an emergency/limit switch
if (zeroSwitchInstalled && zeroInterpolationDone) {
    bool anyCollision = false;

    for (int i = 0; i < numMotors; i++) {
        if (!motors[i]->getZeroSwitchPinState()) { // Switch active =
collision/emergency zone
            anyCollision = true;

            // Show message only once per collision event to avoid flooding Serial
            if (!zeroHandlingZoneDone) {
                DBGL("Motor ");
                DBG(i + 1);
                DBG(" collided with Emergency (here zero) swith. please pull the Motor
out from emergency zone ");
            }

            motors[i]->forceStop();
        }
    }

    if (anyCollision) {
        zeroHandlingZoneDone = true; // Latch until switches are cleared
        zeroInterpolationDone = false; // Require re-zero interpolation after
collision
        return; // Go back to start of loop, skip
Serial/controls
    } else {
        // All switches clear: allow future collisions to print message again
        zeroHandlingZoneDone = false;
    }
}

```

```

}

/* ----- SERIAL COMMAND PARSING ----- */
if (Serial.available() > 0) {
    DBGL();
    DBGL("m or M = Manual, e.g: P 30 60, parallel movement, eg: S 30 60");

    String input = Serial.readStringUntil('\n');

    // Parse command: "P 30 60" or "S 30 60" or "M"
    String tokens[numMotors + 1];
    int tokenCount = 0;
    int startIndex = 0;

    for (int i = 0; i < input.length(); i++) {
        if (input.charAt(i) == ' ' || i == input.length() - 1) {
            tokens[tokenCount] = input.substring(startIndex, (i == input.length() - 1) ? i + 1 : i);
            startIndex = i + 1;
            tokenCount++;
            if (tokenCount == numMotors + 1) break;
        }
    }

    /* --- MANUAL MODE COMMAND --- */
    if (tokens[0] == "M" || tokens[0] == "m") {
        DBG("Manual mode active for all motors");
        for (int i = 0; i < numMotors; i++) {
            motors[i]->initializeMotorInManual();
            motors[i]->setMode(operationMode::MANUAL_MODE);
            motors[i]->setStatus(MotorStatus::RUN);
        }
    }

    /* --- MOVEMENT COMMANDS (P/S + SETPOINTS) --- */
    else if (tokenCount == numMotors+1) {
        // Set movement mode
        if (tokens[0] == "P" || tokens[0] == "p")
            for (int i = 0; i < numMotors; i++)
                motors[i]->setMode(operationMode::PARALLEL_MOVEMENT);

        else if (tokens[0] == "S" || tokens[0] == "s")
            for (int i = 0; i < numMotors; i++)
                motors[i]->setMode(operationMode::SEQUENTIAL_MOVEMENT);

        // Process setpoints for each motor
    }
}

```

```

for (int i = 0; i < numMotors; i++) {
    motors[i]->setSetpoint(constrain(tokens[i+1].toDouble(), 0, 360));

    // Check if already at target position
    if (fabs(motors[i]->getError()) < 4*MAX_ACCEPABLE_ERR) {
        motors[i]->initializeMotorInManual();
        motors[i]->setStatus(MotorStatus::STOP_REST);
        DBGL("Already in position. pls enter new position! ");
    }
    else {
        // Start movement to new position
        motors[i]->initializeMotorInAuto();
        motors[i]->setStatus(MotorStatus::RUN);

        // Print motor configuration and status
        DBGL();
        DBG("Motor ");
        DBG(i + 1);
        DBG(" | setpoint: ");
        DBG(motors[i]->getSetpoint());
        DBG(" | Curr Position= ");
        DBG(motors[i]->getPosition());
        DBG(" | Kp= ");
        DBG(motors[i]->getPID()->GetKp());
        DBG(" | Ki= ");
        DBG(motors[i]->getPID()->GetKi());
        DBG(" | Kd= ");
        DBG(motors[i]->getPID()->GetKd());
        DBG(" | MinFeed= ");
        DBG(motors[i]->getMinSpeed());
        DBG(" | MaxFeed= ");
        DBG(motors[i]->getMaxSpeed());
        DBG(" | SampleTime= ");
        DBG(motors[i]->getPidSampleTime());
    }
}
} else {
    DBGL("Invalid input. Please enter setpoints for all motors.");
}
}

/* ----- CONTROL LOGIC ----- */
switch (motors[0]->getMode()) {

    case operationMode::MANUAL_MODE:

```



```

case operationMode::PARALLEL_MOVEMENT:
    // Process all motors simultaneously
    for (int i = 0; i < numMotors; i++) {
        if (motors[i]->getStatus() != MotorStatus::RUN)
            continue; // Skip motor if not in RUN status

        if (fabs(motors[i]->getError()) > MAX_ACCEPABLE_ERR && !motors[i]-
>getInterpolationStatus()) {
            // Still moving to target
            motors[i]->getPID()->Compute();
            if (fabs(motors[i]->getError()) < 3*MAX_ACCEPABLE_ERR) // Prevent
flooding screen
                motors[i]->printDebug();
        } else {
            // Target reached
            motors[i]->setOutput(0);
            motors[i]->setInterpolationStatus(true);
            motors[i]->setStatus(MotorStatus::STOP_REST);
        }

        motors[i]->setMotor();
    }
    break;

case operationMode::SEQUENTIAL_MOVEMENT:
    // Process motors one at a time in sequence
    for (int i = 0; i < numMotors;) {
        if (motors[i]->getStatus() != MotorStatus::RUN) {
            i++; // Increment before skipping to avoid infinite loop
            continue;
        }

        if (fabs(motors[i]->getError()) > MAX_ACCEPABLE_ERR && !motors[i]-
>getInterpolationStatus()) {
            // Motor still moving
            motors[i]->getPID()->Compute();
            motors[i]->setMotor();
            if (fabs(motors[i]->getError()) < 3*MAX_ACCEPABLE_ERR) // Prevent
flooding screen
                motors[i]->printDebug();
            break; // CRUCIAL: Don't block loop - allow emergency/watchdog
monitoring
        } else {
            // Motor reached target - move to next
            motors[i]->setOutput(0);

```

```

        motors[i]->setMotor();
        motors[i]->setInterpolationStatus(true);
        motors[i]->setStatus(MotorStatus::STOP_REST);
        i++;
    }
}
break;
}

/* ----- SOFTWARE WATCHDOG (AFTER CONTROL) ----- */
softwareWatchdogCheck();
}

//Motor.h
#ifndef MOTOR_H
#define MOTOR_H
#include <Arduino.h>
#include <hardware/watchdog.h>
#include <pico/bootrom.h>
#include <pico/time.h>
// #include <pico/stdlib.h>
#include <hardware/pwm.h>
#include "motor_config.h"

#if DEBUG_SERIAL
    #define DBG(x) Serial.print(x)
    #define DBGL(x) Serial.println(x)
#else
    #define DBG(x)
    #define DBGL(x)
#endif

#if USE_CLASSICAL_PID
    #include <PID_v1.h>
    using PID_t = PID;
#else
    #include "PID_vel.h"
    using PID_t = PID_vel;
#endif

enum class MotorStatus {
    STOP_REST, HOLD, RUN, ZERO_INTERPOL, NORM_INTERPOL, TIME_OUT, ACCELERATION,
    DECELERATION
};

```

```

enum class operationMode {
    MANUAL_MODE,
    PARALLEL_MOVEMENT,
    SEQUENTIAL_MOVEMENT
};

class motor {
public:
    motor(int id, int pwm_pin, int forward_pin, int backward_pin,
          int encoderA_pin, int encoderB_pin, int potentiometer_jogging_pin,
          int zero_switch_pin, double Kp, double Ki, double Kd, int pidSampleTime,
          volatile unsigned EncoderFilterMicroSecond);
    ~motor();

    void begin();
    void updateEncoder();
    void setMotor(); // Uses internal output value
    void printDebug();
    void printMode() const;

    void setMode(operationMode newMode) { mode = newMode; };
    void initializeMotorInAuto() ;
    void initializeMotorInManual() ;
    void initializePwm();

    int getId() { return id; };
    operationMode getMode() const { return mode; };
    void setSetpoint(double inputValue) { set_value = inputValue; };
    void setStatus(MotorStatus statusValue) { motorStatus = statusValue; };
    double getSetpoint() { return set_value; };
    MotorStatus getStatus() { return motorStatus; };
    volatile long getEncoderCount() { return EncoderCounter; };
    volatile long setEncoderCount(long EncoderValue) { return EncoderCounter =
EncoderValue; };
    void setOutput(double value) { output = value; };
    double getOutput() { return output; };
    double setMaxSpeed ( double maxSpeed) { return motorMaxSpeed = maxSpeed; };
    double setMinSpeed ( double minSpeed) { return motorMinSpeed = minSpeed; };
    double getMaxSpeed () { return motorMaxSpeed; };
    double getMinSpeed () { return motorMinSpeed; };
    bool getInterpolationStatus() { return interpolationSuccessfullyDone; };
    void setInterpolationStatus(bool status) { interpolationSuccessfullyDone =
status; };
    bool getZeroInterpolation() { return zeroInterpolationDone; };

```

```

    bool getZeroSwitchPinState() { return zeroSwitchTriggered =
digitalRead(zero_switch_pin); };
    bool zeroInterpolationFunction();
    int getFilterCount() { return filterCount; };
    int getPwm_pin() { return pwm_pin; };
    int getForward_pin() { return forward_pin; };
    int getBackward_pin() { return backward_pin; };
    int getPidSampleTime() { return pidSampleTime; };
    double getPosition();
    double getError();
    void forceStop();
    void resetControlState();
    PID_t* getPID() { return PID_FUNCTION; }

    // static ISRs (one pair per potential motor)
    static void handleInterruptA0();
    static void handleInterruptB0();
    static void handleInterruptA1();
    static void handleInterruptB1();
    static void handleInterruptA2();
    static void handleInterruptB2();
    static void handleInterruptA3();
    static void handleInterruptB3();
    static void handleInterruptA4();
    static void handleInterruptB4();

private:
    int id;
    int pwm_pin;
    int forward_pin;
    int backward_pin;
    int encoderA_pin;
    int encoderB_pin;
    int potentiometer_jogging_pin;
    int zero_switch_pin;

    MotorStatus motorStatus;
    operationMode mode;
    int EncoderCountsPerRevolution;
    int pidSampleTime;

    bool interpolationSuccessfullyDone;
    bool zeroInterpolationDone;
    bool zeroSwitchTriggered;
    float pwmDutyCycle = 250.0;

```

```

double motorMaxSpeed, motorMinSpeed;
double Kp, Ki, Kd;    /// Ki = Ti, Kd = Td if PID velocity uses
double set_value;
double error;
double output;

volatile uint8_t prevState, currState;
volatile unsigned long lastEncoderMicros;
volatile long EncoderCounter;
double current_position;
volatile unsigned EncoderFilterMicroSecond;

double potValue, previous_potValue;
unsigned long last_time;
int filterCount, numberOfCoefficient;
int potFilterCounter;

PID_t* PID_FUNCTION;
static motor* motorInstances[MAX_NUM_MOTORS];
};

#endif

//motor.cpp

#include "motor.h"

// define motorInstances array
motor* motor::motorInstances[MAX_NUM_MOTORS] = { nullptr };

motor::motor(int id, int pwm_pin, int forward_pin, int backward_pin,
             int encoderA_pin, int encoderB_pin, int potentiometer_jogging_pin,
             int zero_switch_pin, double Kp, double Ki, double Kd, int
pidSampleTime, volatile unsigned EncoderFilterMicroSecond)
: id(id),
  pwm_pin(pwm_pin),
  forward_pin(forward_pin),
  backward_pin(backward_pin),
  encoderA_pin(encoderA_pin),
  encoderB_pin(encoderB_pin),
  potentiometer_jogging_pin(potentiometer_jogging_pin),
  zero_switch_pin(zero_switch_pin),

```

```

    Kp(Kp), Ki(Ki), Kd(Kd),
    motorMaxSpeed(MAX_SPEED), motorMinSpeed(MOTOR_DEAD_BAND_SPEED),
    EncoderCountsPerRevolution(COUNT_PER_REVOLUTION),
    pidSampleTime(pidSampleTime),
    pwmDutyCycle(250.0),
    interpolationSuccessfullyDone(true),
    zeroInterpolationDone(false),
    zeroSwitchTriggered(false),
    prevState(0),
    currState(0),
    lastEncoderMicros(0),
    EncoderCounter(0),
    set_value(0),
    current_position(0),
    EncoderFilterMicroSecond(EncoderFilterMicroSecond)
{
    motorStatus = MotorStatus::STOP_REST;
    mode = operationMode::SEQUENTIAL_MOVEMENT;

    pinMode(pwm_pin, OUTPUT);
    pinMode(forward_pin, OUTPUT);
    pinMode(backward_pin, OUTPUT);
    pinMode(encoderA_pin, INPUT_PULLUP);
    pinMode(encoderB_pin, INPUT_PULLUP);
    pinMode(potentiometer_jogging_pin, INPUT);
    pinMode(zero_switch_pin, INPUT);

    initializePwm();

    PID_FUNCTION = new PID_t(&current_position, &output, &set_value, Kp, Ki, Kd,
P_ON_E, DIRECT);

    PID_FUNCTION->SetSampleTime(pidSampleTime);
    PID_FUNCTION->SetMode(AUTOMATIC);
    PID_FUNCTION->SetOutputLimits(-motorMaxSpeed, motorMaxSpeed);

    if (id < MAX_NUM_MOTORS) motorInstances[id] = this;
}

motor::~motor() {
    delete PID_FUNCTION;
}

void motor::begin() {
    // Attach interrupts only for this motor id

```



```

    switch (id) {
        case 0:
            attachInterrupt(digitalPinToInterrupt(encoderA_pin),
motor::handleInterruptA0, CHANGE);
            attachInterrupt(digitalPinToInterrupt(encoderB_pin),
motor::handleInterruptB0, CHANGE);
            break;
        case 1:
            attachInterrupt(digitalPinToInterrupt(encoderA_pin),
motor::handleInterruptA1, CHANGE);
            attachInterrupt(digitalPinToInterrupt(encoderB_pin),
motor::handleInterruptB1, CHANGE);
            break;
        case 2:
            attachInterrupt(digitalPinToInterrupt(encoderA_pin),
motor::handleInterruptA2, CHANGE);
            attachInterrupt(digitalPinToInterrupt(encoderB_pin),
motor::handleInterruptB2, CHANGE);
            break;
        case 3:
            attachInterrupt(digitalPinToInterrupt(encoderA_pin),
motor::handleInterruptA3, CHANGE);
            attachInterrupt(digitalPinToInterrupt(encoderB_pin),
motor::handleInterruptB3, CHANGE);
            break;
        case 4:
            attachInterrupt(digitalPinToInterrupt(encoderA_pin),
motor::handleInterruptA4, CHANGE);
            attachInterrupt(digitalPinToInterrupt(encoderB_pin),
motor::handleInterruptB4, CHANGE);
            break;
    }
}

// create ISR pairs that only call the corresponding motor->updateEncoder()
#define MAKE_ISR_PAIR(N) \
    void motor::handleInterruptA##N() { if (motor::motorInstances[N]) \
motor::motorInstances[N]->updateEncoder(); } \
    void motor::handleInterruptB##N() { if (motor::motorInstances[N]) \
motor::motorInstances[N]->updateEncoder(); }

MAKE_ISR_PAIR(0)
MAKE_ISR_PAIR(1)
MAKE_ISR_PAIR(2)
MAKE_ISR_PAIR(3)

```

```

MAKE_ISR_PAIR(4)

void motor::updateEncoder() {
    #if USE_ENCODER_FILTER    // Filter: ignore too-fast interrupts
        unsigned long now = micros();
        unsigned long delta = now - lastEncoderMicros;

        if (delta < EncoderFilterMicroSecond) {
            return;    // ignore noise
        }

        lastEncoderMicros = now;    // update timestamp for next interrupt
    #endif

    int MSB = digitalRead(encoderA_pin);
    int LSB = digitalRead(encoderB_pin);
    currState = (MSB << 1) | LSB;
    int sum = (prevState << 2) | currState;

    switch (sum) {
        case 0b1101: case 0b0100: case 0b0010: case 0b1011:
            EncoderCounter--;
            break;
        case 0b1110: case 0b0111: case 0b0001: case 0b1000:
            EncoderCounter++;
            break;
    }

    prevState = currState;
}

double motor::getPosition() {
    noInterrupts();
    current_position = (EncoderCounter * 360.0) / (long)
EncoderCountsPerRevolution;
    interrupts();
    return current_position;
}

double motor::getError() {
    error = set_value - getPosition();
    return error;
}

```

```

void motor::initializePwm() {
    // Keep your original Raspberry Pi Pico PWM initialization logic
    gpio_set_function(pwm_pin, GPIO_FUNC_PWM);

    int slice_pan = pwm_gpio_to_slice_num(pwm_pin);
    pwm_set_wrap(slice_pan, 1023);

    // set clock divider appropriate (original values preserved)
    pwm_set_clkdiv(slice_pan, 129.8f); // example to ~1kHz as you had

    pwm_set_enabled(slice_pan, true);

    pwm_set_gpio_level(pwm_pin, (int)pwmDutyCycle);
}

void motor::initializeMotorInAuto() {
    interpolationSuccessfullyDone = false;
    filterCount=0;
    potFilterCounter=0;
}

void motor::initializeMotorInManual() {
    previous_potValue = analogRead(potentiometer_jogging_pin);
    interpolationSuccessfullyDone = true;
    filterCount=0;
    potFilterCounter=0;
}

void motor::setMotor() {
    int pwmVal = map(abs(output), 0, motorMaxSpeed, motorMinSpeed, motorMaxSpeed);
    // int pwmVal = constrain(abs(output), motorMinSpeed, motorMaxSpeed);
    if (output > 0) {
        pwm_set_gpio_level(pwm_pin, pwmVal);
        digitalWrite(forward_pin, HIGH);
        digitalWrite(backward_pin, LOW);
    } else if (output < 0) {
        pwm_set_gpio_level(pwm_pin, pwmVal);
        digitalWrite(forward_pin, LOW);
        digitalWrite(backward_pin, HIGH);
    } else {
        pwm_set_gpio_level(pwm_pin, 0);
        digitalWrite(forward_pin, LOW);
        digitalWrite(backward_pin, LOW);
    }
}

```

```

void motor::printDebug() {
    DBGL();
    DBG(" id: ");
    DBG(id);
    DBG(" | Setpoint: ");
    DBG(getSetpoint());
    DBG(" | Encoder: ");
    DBG(getEncoderCount());

    DBG(" | Position: ");
    DBG(getPosition());
    DBG(" | Error: ");
    DBG(getError());
    DBG(" | Output: ");
    DBG(getOutput());
}

void motor::printMode() const {
    switch (mode) {
        case operationMode::MANUAL_MODE:
            DBG("Mode: Manual");
            break;
        case operationMode::PARALLEL_MOVEMENT:
            DBG("Mode: Parallel Movement");
            break;
        case operationMode::SEQUENTIAL_MOVEMENT:
            DBG("Mode: Sequential Movement");
            break;
    }
}

bool motor::zeroInterpolationFunction() {
    //define later
    return false;
}

void motor::forceStop() {
    set_value = 0; // may earse afterwards if i used this outside of emergency
function
    output = 0;
    motor::setMotor();
    EncoderCounter = 0;
    motorStatus = MotorStatus::STOP_REST;
}

```

```

void motor::resetControlState() {
    interpolationSuccessfullyDone = false;
    zeroInterpolationDone = false;
    output = 0;
    set_value = 0;
    EncoderCounter = 0;

    if (PID_FUNCTION) {
        PID_FUNCTION->SetMode(MANUAL);    // stop PID, clear ITerm
        PID_FUNCTION->SetMode(AUTOMATIC); // restart clean
    }
}

/* PID_vel.cpp */

#include "PID_vel.h"

//Constructor *****
PID_vel::PID_vel(double* Feedback, double* Output, double* Setpoint,
                double Kp, double Ti, double Td, int POn, int ControllerDirection)
{
    myOutput = Output;
    myPosition = Feedback;
    mySetpoint = Setpoint;
    inAuto = false;
    controllerDirection = ControllerDirection; // Store but not used in velocity
    form
    error_1 = 0;
    error_2 = 0;
    outputSum = 0;

    PID_vel::SetOutputLimits(-255, 255);    //default output limit
    corresponds to                          //the pico pwm limits

    SampleTime = 100;                      //default Controller Sample Time is 0.1
    seconds
    PID_vel::SetTunings(Kp, Ti, Td);

    lastTime = millis()-SampleTime;
}

PID_vel::PID_vel(double* Feedback, double* Output, double* Setpoint,
                double Kp, double Ti, double Td)

```

```

        :PID_vel::PID_vel(Feedback, Output, Setpoint, Kp, Ti, Td, 0, DIRECT) //
Default values for unused params
{
}

void PID_vel::SetTunings(double Kp, double Ti, double Td)
{
    if (Kp<0 || Ti<0 || Td<0) return;

    dispKp = Kp, dispKi = Kp/Ti, dispKd = Kp*Td;
    //Calculate constances in order not to do mathematics in each Compute call
    // output += Kp * (1 + Td/T) * error + Kp * (T/Ti - 1 - 2*Td/T) * error_1 +
Kp * Td/T * error_2 ); or
    // output += constant1 * error + constant2 * error_1 + constant3 * error_2 ;
    constant1 = Kp * (1 + Td/SampleTime);
    if (Ti != 0) constant2 = Kp * (SampleTime/Ti - 1 - 2*Td/SampleTime);
    else constant2 = 0;
    constant3 = Kp * Td/SampleTime;
}

/* SetSampleTime(...) *****
 * sets the period, in Milliseconds, at which the calculation is performed
 ***** */
void PID_vel::SetSampleTime(int NewSampleTime)
{
    if (NewSampleTime > 0)
    {
        SampleTime = (unsigned long)NewSampleTime;
    }
}

void PID_vel::SetOutputLimits(double Min, double Max)
{
    if(Min >= Max) return;
    outMin = Min;
    outMax = Max;

    if(inAuto)
    {
        if(*myOutput > outMax) *myOutput = outMax;
        else if(*myOutput < outMin) *myOutput = outMin;

        if(outputSum > outMax) outputSum= outMax;
        else if(outputSum < outMin) outputSum= outMin;
    }
}

```



```

    }
}
void PID_vel::SetMode(int Mode)
{
    bool newAuto = (Mode == AUTOMATIC);
    if(newAuto && !inAuto)
    { /*we just went from manual to auto*/
        PID_vel::Initialize(); // if the mode is changed from Manual to
Automatic, initialize the controller
    }
    inAuto = newAuto;
}
/* Initialize()*****
 * does all the things that need to happen to ensure a bumpless transfer
 * from manual to automatic mode.
 *****/
void PID_vel::Initialize()
{
    outputSum = 0;
    error_1 = 0;
    error_2 = 0;
}
bool PID_vel::Compute()
{
    if(!inAuto) return false;
    unsigned long now = millis();
    unsigned long timeChange = (now - lastTime);
    if(timeChange>=SampleTime)
    {
        double feedback = *myPosition;
        double error = *mySetpoint - feedback;
        double output = 0;
        // PID calculations diffrent method
        //outputSum += Kp * ((1 + Ki + Kd) * error - (1 + 2 * Kd) * error_1 + Kd *
error_2 ); // PID formula right point integration
        //outputSum += Kp * ( (1 + Kd) * error + (Ki - 1 - 2*Kd) * error_1 + Kd *
error_2 ); // PID formula left point integration
        outputSum += constant1 * error + constant2 * error_1 + constant3 * error_2 ;

        //if(outputSum > outMax) outputSum= outMax;
        //else if(outputSum < outMin) outputSum= outMin;
        output = constrain(outputSum, outMin, outMax);
        *myOutput = output;
        error_2 = error_1;
        error_1 = error;
    }
}

```

```

    /*if (output = 0) {
        outputSum = 0;
        error_1 = 0;
        error_1 = 0;
    } */

    lastTime = now;
    return true;
}
else return false;
}

double PID_vel::GetKp(){ return constant1; }
double PID_vel::GetKi(){ return constant2;}
double PID_vel::GetKd(){ return constant3;}
int PID_vel::GetMode(){ return inAuto ? AUTOMATIC : MANUAL;}

```

```

/**
 * @file system_safety.cpp
 * @brief Implementation of emergency and watchdog safety systems
 */

#include "system_safety.h"
#include <Arduino.h>

/**
 * @brief Emergency stop handler
 *
 * Key changes (02.02.2026):
 * - Emergency pulled up externally with 120k (internal pull-up didn't work)
 * - Normal condition: LOW (through emergency push button)
 * - If wire breaks/cuts: goes HIGH through pull-up resistor (emergency state)
 * - Logic inverted from previous program version
 *
 * Safety feature: Wire disconnect = Emergency (fail-safe design)
 */
void checkAndHandleEmergency() {

    // Check emergency input (HIGH = emergency) - polling method
    if (digitalRead(EMERGENCY_PIN) == HIGH) {
        emergencyTriggered = true;
    }

    // Emergency triggered - stop all motors (one-time action)

```

```

if (emergencyTriggered && !emergencyHandlingDone) {

    DBGL("[EMERG] STOP");

    for (int i = 0; i < numMotors; i++) {
        motors[i]->getPID()->SetMode(MANUAL); // Disable PID
        motors[i]->forceStop();              // Emergency brake
    }

    emergencyHandlingDone = true;
    return;
}

// Emergency cleared - restore normal operation
if (emergencyTriggered && digitalRead(EMERGENCY_PIN) == LOW) {

    DBGL("[EMERG] CLEARED");

    // Re-enable PID for all motors
    for (int i = 0; i < numMotors; i++)
        motors[i]->getPID()->SetMode(AUTOMATIC);

    // Reset emergency flags
    emergencyTriggered = false;
    emergencyHandlingDone = false;

    // Force re-zero calibration (essential for industrial safety)
    zeroInterpolationDone = false;
}
}

/**
 * @brief Software watchdog - prevents infinite loops or stalls
 *
 * Key changes (02.02.2026):
 * - Moved AFTER control logic in main loop
 * - Ensures entire loop cycle completed before checking
 *
 * If loop stalls (millis() - lastLoopKick > SW_WATCHDOG_MS):
 * - Logs warning
 * - Enables 200ms hardware watchdog
 * - Enters infinite loop
 * - Hardware watchdog triggers reset
 *
 * Important: lastLoopKick must be updated at start of each loop()

```

```

*/
void softwareWatchdogCheck() {
    if ((millis() - lastLoopKick) > SW_WATCHDOG_MS) {
        DBGL("[SW WD] Loop stalled - reset");
        watchdog_enable(200, false); // Short timeout, no pause
        delay(500);                  // Ensure message sent
        while (true) {}              // Wait for hardware reset
    }
}

```

```

/**
 * @file zero_calibration.cpp
 * @brief Implementation of zero position calibration functions
 */

#include "zero_calibration.h"
#include <Arduino.h>

// State machine variables for non-blocking mode
ZeroState zeroState = ZERO_IDLE;
int zeroMotorIndex = 0;
unsigned long zeroTimestamp = 0;

/**
 * @brief Blocking zero search - processes all motors sequentially
 *
 * For each motor that needs calibration:
 * 1. Move backward until zero switch triggered
 * 2. Stop motor
 * 3. Move forward for 250ms to park position
 * 4. Reset encoder count to 0
 *
 * Monitors emergency during process and updates watchdog.
 */
bool zeroSearchMotors() {
    DBGL("Starting zero interpolation for all motors...");
    zeroInterpolationDone = false;

    // Set all motors to ZERO_INTERPOL mode
    for (int i = 0; i < numMotors; i++) {
        motors[i]->setMinSpeed(0);
        motors[i]->setStatus(MotorStatus::ZERO_INTERPOL);
    }
}

```

```

    motors[i]->getPID()->SetMode(MANUAL);    // stop PID winding up integral part
inside-- refer to pid code
}

// Process each motor sequentially
for (int i = 0; i < numMotors; i++) {
    if (!motors[i]->getZeroInterpolation()) {
        DBG("Motor ");
        DBG(i);
        DBGL(" searching for zero...");

        // Move backward until zero switch is triggered
        while(motors[i]->getZeroSwitchPinState()) {
            // Check for emergency during calibration
            extern void checkAndHandleEmergency();
            checkAndHandleEmergency();
            if (emergencyTriggered)
                return false;
            motors[i]->setOutput(-ZERO_INTERPOLATION_FEED_MIN);
            motors[i]->setMotor();
            watchdog_update(); // Prevent watchdog timeout
        }
        motors[i]->setOutput(0);
        motors[i]->setMotor();
        delay(100); // debounce delay

        // Move back forward slowly until zero switch is released
        // Alternative approach: just go back a little bit (current implementation)
        /* while(!motors[i]->getZeroSwitchPinState()) {
            motors[i]->setOutput(ZERO_INTERPOLATION_FEED_MAX);
            motors[i]->setMotor();
            watchdog_update(); // Prevent watchdog timeout
            DBGL(" Motor roll is backing to park state...");
        } */

        motors[i]->setOutput(ZERO_INTERPOLATION_FEED_MAX);
        motors[i]->setMotor();
        delay(250); // get a little distance from zero switch
        motors[i]->setOutput(0);
        motors[i]->setMotor();
        delay(100); // debounce delay
        motors[i]->setIntorpolationStatus(true);
        motors[i]->setEncoderCount(0);
        motors[i]->setStatus(MotorStatus::STOP_REST);
    }
}

```

```

        DBG("Motor ");
        DBG(i);
        DBGL(" zero interpolation done. Encoder reset to 0.");
    }
}

// Restore normal operating parameters for all motors
zeroInterpolationDone = true;
for (int i = 0; i < numMotors; i++){
    motors[i]->setMinSpeed(MOTOR_DEAD_BAND_SPEED);
    motors[i]->getPID()->SetMode(AUTOMATIC);
}

DBGL("Zero interpolation completed for all motors.");
return true;
}

/**
 * @brief Non-blocking zero calibration - state machine version
 *
 * Must be called repeatedly from main loop. Processes one state per call.
 * Allows emergency monitoring and prevents watchdog timeouts during
 * long calibration sequences.
 *
 * Industrial-style implementation: safe, maintainable, non-blocking.
 */
void zeroSearchMotorsNonBlocking() {

    switch (zeroState) {

        case ZERO_IDLE:
            // Initialize calibration sequence
            zeroInterpolationDone = false;
            zeroMotorIndex = 0;
            zeroState = ZERO_PREPARE;
            DBGL("Zero interpolation started (non-blocking)");
            break;

        case ZERO_PREPARE:
            // Setup current motor for calibration
            motors[zeroMotorIndex]->getPID()->SetMode(MANUAL);
            motors[zeroMotorIndex]->setMinSpeed(0);
            motors[zeroMotorIndex]->setStatus(MotorStatus::ZERO_INTERPOL);
            DBGL("Zero preparation (non-blocking)");
            zeroState = ZERO_BACKWARD;
    }
}

```



```

        break;

    case ZERO_BACKWARD:
        // Move backward until zero switch triggered
        if (!motors[zeroMotorIndex]->getZeroSwitchPinState()) {
            // Switch triggered - stop and transition to forward
            motors[zeroMotorIndex]->setOutput(0);
            motors[zeroMotorIndex]->setMotor();
            zeroTimestamp = millis();
            zeroState = ZERO_FORWARD;
        } else {
            // Keep moving backward
            motors[zeroMotorIndex]->setOutput(-ZERO_INTERPOLATION_FEED_MIN);
            motors[zeroMotorIndex]->setMotor();
            DBGL("ZERO_BACKWARD (non-blocking)");
        }
        break;

    case ZERO_FORWARD:
        // Move forward for 250ms to park position
        if (millis() - zeroTimestamp < 250) {
            motors[zeroMotorIndex]->setOutput(ZERO_INTERPOLATION_FEED_MAX);
            motors[zeroMotorIndex]->setMotor();
            DBGL("ZERO_FORWARD (non-blocking)");
        } else {
            // Park time elapsed - stop motor
            motors[zeroMotorIndex]->setOutput(0);
            motors[zeroMotorIndex]->setMotor();
            zeroState = ZERO_FINALIZE;
        }
        break;

    case ZERO_FINALIZE:
        // Reset encoder and mark motor as calibrated
        motors[zeroMotorIndex]->setEncoderCount(0);
        motors[zeroMotorIndex]->setIntorpolationStatus(true);
        motors[zeroMotorIndex]->setStatus(MotorStatus::STOP_REST);
        DBG("Motor ");
        DBG(zeroMotorIndex);
        DBGL(" zero done");

        zeroMotorIndex++;

        // Check if all motors calibrated
        if (zeroMotorIndex >= numMotors) {

```

```

        zeroState = ZERO_DONE;
    } else {
        zeroState = ZERO_PREPARE; // Move to next motor
    }
    break;

case ZERO_DONE:
    // Restore normal operating parameters for all motors
    for (int i = 0; i < numMotors; i++){
        motors[i]->setMinSpeed(MOTOR_DEAD_BAND_SPEED);
        motors[i]->getPID()->SetMode(AUTOMATIC);
    }
    zeroInterpolationDone = true;
    zeroState = ZERO_IDLE;
    DBGL("Zero interpolation completed (non-blocking)");
    break;
}
}

```

```

/**
 * @file system_safety.h
 * @brief Emergency stop and watchdog safety systems
 *
 * Implements two critical safety features:
 * 1. Emergency stop monitoring (hardware input)
 * 2. Software watchdog (loop stall detection)
 *
 * Emergency pin uses external pull-up (120k). Normal = LOW, Emergency = HIGH.
 * If emergency wire breaks or disconnects, system enters safe emergency state.
 */

#ifndef SYSTEM_SAFETY_H
#define SYSTEM_SAFETY_H

#include "motor.h"
#include "motor_config.h"

// External references to shared state (defined in main.cpp)
extern motor** motors;
extern int numMotors;
extern bool emergencyTriggered;
extern bool emergencyHandlingDone;
extern bool zeroInterpolationDone;

```

```

extern unsigned long lastLoopKick;
extern const uint8_t EMERGENCY_PIN;

/**
 * @brief Check emergency input and handle state transitions
 *
 * Three states:
 * 1. Normal operation: EMERGENCY_PIN = LOW
 * 2. Emergency triggered: EMERGENCY_PIN = HIGH -> Stop all motors
 * 3. Emergency cleared: EMERGENCY_PIN goes LOW -> Re-enable system, force re-
zero
 *
 * Must be called frequently from main loop.
 */
void checkAndHandleEmergency();

/**
 * @brief Software watchdog - detect loop stalls
 *
 * Monitors time since last loop kick (lastLoopKick timestamp).
 * If loop doesn't execute for > SW_WATCHDOG_MS, forces system reset
 * by enabling short hardware watchdog without updates.
 *
 * Critical: Call AFTER all control logic to ensure loop completed.
 */
void softwareWatchdogCheck();

#endif // SYSTEM_SAFETY_H

```

```

/**
 * @file zero_calibration.h
 * @brief Zero position calibration for DC servo motors
 *
 * Provides both blocking and non-blocking zero calibration methods.
 * Zero calibration locates the mechanical zero position using limit switches
 * and resets encoder counts. Essential for industrial applications,
 * especially after emergency stops.
 */

#ifndef ZERO_CALIBRATION_H
#define ZERO_CALIBRATION_H

#include "motor.h"

```

```

#include "motor_config.h"

// External references to shared state (defined in main.cpp)
extern motor** motors;
extern int numMotors;
extern bool zeroInterpolationDone;
extern bool emergencyTriggered;

/**
 * @brief Blocking zero calibration for all motors
 *
 * Sequentially moves each motor backward to trigger zero switch,
 * then moves forward slightly to park position. Sets encoder to zero.
 * Uses while loops - blocks program execution until complete.
 *
 * @return true if successful, false if emergency triggered
 */
bool zeroSearchMotors();

/**
 * @brief Non-blocking zero calibration using state machine
 *
 * Industrial-style state machine approach. Must be called repeatedly
 * from main loop. Allows emergency monitoring and watchdog updates
 * during calibration process.
 *
 * States: IDLE -> PREPARE -> BACKWARD -> FORWARD -> FINALIZE -> DONE
 */
void zeroSearchMotorsNonBlocking();

// State machine states for non-blocking calibration
enum ZeroState {
    ZERO_IDLE,        // Not running
    ZERO_PREPARE,     // Setup motor for calibration
    ZERO_BACKWARD,    // Moving backward to find zero switch
    ZERO_FORWARD,     // Moving forward to park position
    ZERO_FINALIZE,    // Reset encoder and prepare next motor
    ZERO_DONE         // All motors calibrated
};

// State machine variables (shared across calls)
extern ZeroState zeroState;
extern int zeroMotorIndex;
extern unsigned long zeroTimestamp;

```

```
#endif // ZERO_CALIBRATION_H
```

```
/* PID_vel.h */
#ifndef PID_vel_h
#define PID_vel_h
#define LIBRARY_VERSION 1.1.1
#include <Arduino.h>
#include "motor_config.h"

class PID_vel
{
    //Constants used in some of the functions below
#define AUTOMATIC 1
#define MANUAL 0
#define DIRECT 0
#define REVERSE 1
#define P_ON_M 0
#define P_ON_E 1
public:
    //commonly used functions
    *****
    PID_vel(double*, double*, double*,          // * constructor. links the PID to
the Input, Output, and
        double, double, double, int, int);// Setpoint. Initial tuning
parameters are also set here.
                                                // (overload for specifying
proportional mode)
    PID_vel(double*, double*, double*,          // * constructor. links the PID to
the Input, Output, and
        double, double, double);// Setpoint. Initial tuning parameters are
also set here.
                                                // (overload for specifying
proportional mode)

    void SetMode(int Mode);                    // * sets PID to either Manual (0) or
Auto (non-0)

    bool Compute();                            // * performs the PID calculation. it
should be
                                                // called every time loop() cycles.
                                                //
                                                //

```

```

void SetOutputLimits(double, double); // *
//
void SetTunings(double, double, // *
               double);
void SetSampleTime(int);        // * sets the frequency, in
Millisecons, with which        // the PID calculation is
performed. default is 100 ms

//End*****
//Display functions
*****

double GetKp();                // These functions query the pid for interal
values.
double GetKi();                // they were created mainly for the pid front-end,
double GetKd();                // where it's important to know what is actually
int GetMode();                 // inside the PID.//

private:
void Initialize();

double dispKp;                 // * we'll hold on to the tuning parameters in user-
entered
double dispKi;                 // format for display purposes
double dispKd;                 //
// output += Kp * (1 + Td/T) * error + Kp * (T/Ti - 1 - 2*Td/T) * error_1
+ Kp * Td/T * error_2 )
double constant1;              // * Kp * (1 + Td/pidSampleTime)
double constant2;              // * Kp * (T/Ti - 1 - 2*Td/T)
double constant3;              // * Kp * Td/T

int controllerDirection;

double *myPosition;            // * Pointers to the Input, Output, and
Setpoint variables
double *myOutput;              // This creates a hard link between the
variables and the
double *mySetpoint;            // PID, freeing the user from having to
constantly tell us
// what these values are. with pointers
we'll just know.

unsigned long lastTime;
double outputSum, error_1, error_2;

```

```

    unsigned long SampleTime;
    double outMin, outMax;
    bool inAuto, pOnE;
};

#endif

```

```

/* motor_config.h */

#pragma once

/* ===== PID SELECTION ===== */
#define USE_CLASSICAL_PID 1 // 1 = PID_v1, 0 = PID_vel

/* ===== FEATURE SWITCHES ===== */
#define USE_ZERO_NON_BLOCKING 1
#define USE_ENCODER_FILTER 1
#define DEBUG_SERIAL 1

/* ===== WATCHDOG ===== */
constexpr uint32_t HW_WATCHDOG_MS = 5000; // hardware watchdog timeout (ms)
constexpr uint32_t SW_WATCHDOG_MS = 3000; // software watchdog timeout (ms)

/* ===== PID TUNING (GLOBAL DEFAULTS) ===== */
#if USE_CLASSICAL_PID
    constexpr double KP = 5.0;
    constexpr double Ki_Ti = 0.0; // Ki
    constexpr double Kd_Td = 0.25; // Kd
    constexpr int PID_SAMPLE_TIME = 10;
#else
    constexpr double KP = 0.9;
    constexpr double Ki_Ti = 9000.0; // Ti
    constexpr double Kd_Td = 0.0; // Td
    constexpr int PID_SAMPLE_TIME = 100;
#endif

/* ===== SYSTEM CONSTANTS ===== */
constexpr int MAX_NUM_MOTORS = 5;
constexpr int COUNT_PER_REVOLUTION = 4350; // ms
constexpr int MOTOR_DEAD_BAND_SPEED = 350;
constexpr int MAX_SPEED = 1023;

```



```

constexpr double MAX_ACCEPABLE_ERR = 0.1;
constexpr int MANUAL_FEED = 50; // percentage 1 - 100% of maximum feed

/* ===== ZERO INTERPOLATION ===== */
constexpr int ZERO_INTERPOLATION_FEED_MIN = 278;
constexpr int ZERO_INTERPOLATION_FEED_MAX = 280;

/* ===== ENCODER FILTER ===== */
constexpr unsigned ENCODER_FILTER_M1 = 50; // µs
constexpr unsigned ENCODER_FILTER_M2 = 20;

```

10. References

Prof. E. de Lucena,, Samuel: Pictures

Prof. Keller, Frieder: Digital Control Theory Script. EIT, H-KA, Karlsruhe, Germany.

Microcontroller Based Applied Digital Control D. Ibrahim
©2006 John Wiley & Sons, Ltd. ISBN: 0-470-86335-8

Brett Beauregard, *Arduino PID Library – Version 1.2.1*, available at:
<https://github.com/br3ttb/Arduino-PID-Library>

David Forrest, *Arduino PID Library – Version 1.2.3*

Repository latest code: <https://bwsyncandshare.kit.edu/apps/files/files/5241844567?dir=/ProjectB>

<https://github.com/peyman74/RaspberryPico-TwoArms-Robot-Positioning>

My email address: Peiman.edalatjoo@proton.me , edpe1011@h-ka.de